

RoadIA — Backend Documentation

Ce livre documente le backend Rust et l'architecture globale du projet RoadIA. Il couvre la structure du projet, les APIs exposées, le modèle de données de la carte, le moteur de simulation (incluant les modèles de changement de voie LC2013 et les trajets pendulaires) et le système de scoring.

L'objectif de cette documentation est de fournir une vision complète des modules, des structures publiques, et des mécanismes internes (comme la gestion des intersections et le modèle de pollution CO2) pour les développeurs et contributeurs.

Architecture du projet RoadIA

Ce document présente la structure globale du projet RoadIA, les responsabilités de chaque composant et les flux de données principaux.

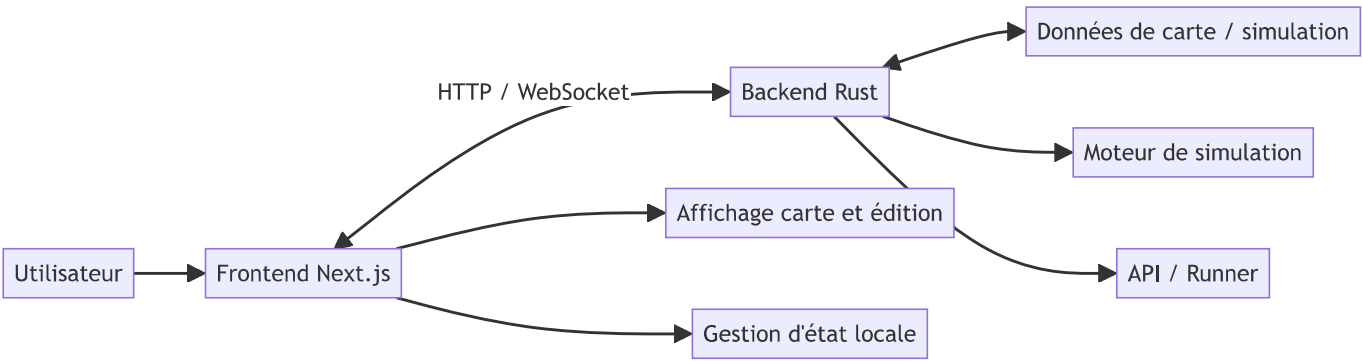
1. Présentation générale

RoadIA est une application web de conception et de simulation de réseau routier en temps réel. Le projet permet de créer une carte, de gérer des infrastructures et d’observer le trafic via un moteur de simulation côté serveur.

L’application est organisée en deux blocs principaux :

- **Frontend** : Next.js et TypeScript (interface utilisateur et édition).
- **Backend** : Rust avec Axum et Tokio (métier, simulation et temps réel).

2. Vue d’ensemble de l’architecture



Le frontend sert de point d’entrée. Il communique avec le backend via HTTP pour les actions classiques et WebSocket pour les mises à jour de simulation en temps réel.

3. Structure du dépôt

Répertoire	Rôle
client/	Interface web, rendu graphique (Pixi.js/Leaflet), interaction utilisateur.
server/	API, simulation, persistance, gestion des WebSockets.

Répertoire	Rôle
<code>docker-compose.yml</code>	Orchestration locale des services.
<code>architecture/</code>	Documentation technique détaillée.

4. Frontend

Le frontend repose sur **Next.js 16**, **React 19** et **TypeScript**.

Responsabilités

- Gestion des cartes (création, chargement, suppression).
- Rendu cartographique et édition interactive.
- Communication temps réel via WebSocket pour l’affichage de la simulation.

Points d’entrée notables

- `client/app/page.tsx` : Accueil et gestion des cartes.
- `client/app/map/[uuid]/page.tsx` : Édition et simulation.

5. Backend

Le backend est écrit en **Rust**, utilisant **Axum** pour le réseau et **Tokio** pour l’asynchronisme.

Responsabilités

- Persistance des données (cartes et exports).
- Orchestration de la simulation de trafic.
- Gestion des flux WebSocket.

Organisation fonctionnelle

- `server/src/api/` : Couche d’exposition réseau.
- `server/src/map/` : Modèle du réseau routier.
- `server/src/simulation/` : Moteur de calcul des déplacements.
- `server/src/scoring/` : Indicateurs de performance.

6. Flux de fonctionnement

Simulation en temps réel

1. Le serveur exécute la boucle de simulation (`step`).
2. Les positions et événements sont calculés.
3. Les données sont diffusées via **WebSocket**.
4. Le frontend met à jour le rendu graphique.

7. Persistance

Les données sont stockées localement dans `server/data/` . Le backend garantit la cohérence des fichiers tandis que le frontend manipule les métadonnées.

API Overview

Le backend expose deux surfaces principales:

- un ensemble de routes HTTP pour créer, enregistrer, charger et configurer les simulations;
- un WebSocket `/ws` pour recevoir l'état courant et envoyer les commandes temps réel.

Pages de référence

- [HTTP handlers](#) : routes HTTP et structure d'état partagée.
- [Runner / Server endpoints](#) : contrôleur, instance de simulation et démarrage du serveur.
- [WebSocket protocol](#) : protocoles JSON échangés avec le client.

Flux général

Le chemin d'utilisation habituel est le suivant:

1. le client crée une simulation via `POST /api/simulations` ou charge une carte enregistrée;
2. le client ouvre `/ws?uuid=<uuid>&token=<token>` ;
3. le serveur envoie l'instantané initial de la carte;
4. le client peut alors démarrer ou arrêter la simulation, éditer la carte ou demander des métriques;
5. le runner publie ensuite les mises à jour de véhicules et, à la fin, le score.

HTTP handlers

Fichier principal: `src/api/runner/handlers.rs`.

Ce chapitre documente l'état partagé du serveur HTTP et les routes qui créent, modifient et chargent les simulations. Les handlers manipulent surtout les instances en mémoire, les fichiers de carte persistés sur disque, et la configuration de la carte.

État partagé

AppState

Contient toutes les simulations actives, indexées par `uuid`. Chaque entrée pointe vers une `SimulationInstance` partagée entre les routes HTTP et le WebSocket.

Requêtes JSON

CustomMapRequest

Décrit une zone géographique à extraire depuis OpenStreetMap.

- `min_lat`, `min_lon` : coin sud-ouest du rectangle.
- `max_lat`, `max_lon` : coin nord-est du rectangle.

SaveMapRequest

Requête de sauvegarde d'une carte existante.

- `uuid` : identifiant de la simulation à sauvegarder.
- `token` : jeton d'accès associé.
- `name` : nom à attribuer à la carte.
- `file_uuid` : identifiant du fichier JSON de destination, s'il existe déjà.

LoadMapRequest

Charge une carte enregistrée à partir de son identifiant de fichier.

RenameMapRequest

Renomme une carte persistée sans recréer de simulation.

DeleteMapRequest

Supprime le fichier JSON d’une carte persistée.

Routes et responsabilités

Route	Fonction	
POST /api/simulations	create_simulation_handler	Créer une simulation par défaut
POST /api/custom_map	create_custom_simulation_handler	Intégrer une carte personnalisée dans une simulation
POST /api/simulations/save-map	save_map_handler	Sauvegarder une carte active
POST /api/simulations/load-map	load_map_handler	Charger une carte personnalisée pour une nouvelle simulation
GET /api/maps	list_maps_handler	Lister toutes les cartes sauvegardées dans la base de données
POST /api/maps/rename	rename_map_handler	Renommer une carte persistée
POST /api/maps/delete	delete_map_handler	Supprimer une carte persistée
GET /api/simulations/:uuid/settings	get_simulation_settings_handler	Lire les paramètres d’une simulation à partir d’une carte
POST /api/simulations/:uuid/settings	update_simulation_settings_handler	Mettre à jour les paramètres d’une simulation à partir d’une carte
run	Démarrage du serveur	Configuration et installation

Route		Fonction
		Axu

Comportement des handlers

create_simulation_handler

Crée une simulation par défaut.

- **Action** : Charge la carte de Lannion et initialise une nouvelle instance de simulation.

create_custom_simulation_handler

Récupère une zone géographique d'OSM et crée une simulation.

- **Entrées** : `State(state)` , `Json(payload)` (coordonnées).
- **Action** : Télécharge les données via Overpass API, les convertit, et initialise une nouvelle instance de simulation.
- **Retour** : `Json` contenant l'UUID et le jeton de la nouvelle instance.

save_map_handler / load_map_handler

Gèrent la persistance des cartes sur le serveur.

- **Action** : Sauvegardent l'état actuel d'une simulation dans un fichier JSON ou chargent un fichier existant pour créer une nouvelle simulation.

rename_map_handler / delete_map_handler

Opérations de maintenance sur les fichiers de cartes.

- **Action** : Renomment ou suppriment physiquement les fichiers JSON du répertoire de données.

list_maps_handler

Liste toutes les cartes enregistrées sur le serveur.

- **Retour** : `Json` contenant la liste des noms et IDs de fichiers.

get_simulation_settings_handler / update_simulation_settings_handler

Accèdent aux réglages de la carte active.

- **Action** : Permettent de consulter ou de modifier les poids du score et les paramètres de budget en cours de simulation.

run

Construit le `Router` , installe les routes HTTP, configure le CORS avec `ALLOWED_ORIGINS` , crée l'état partagé initial puis lance le serveur sur `0.0.0.0:8080` .

À retenir

- Les handlers ne contiennent pas la logique de simulation elle-même.
- Ils servent surtout d'interface de création, de persistance et de configuration.
- Les validations d'entrée sont importantes, en particulier pour les limites géographiques et le calcul des poids du score.

Runner / Server entrypoints

Fichier principal: `src/api/runner/runner.rs`.

Ce chapitre décrit la gestion des simulations en mémoire, les contrôleurs de démarrage/arrêt et le lancement du serveur HTTP + WebSocket.

SimulationController

Petit contrôleur atomique qui indique si une simulation doit tourner.

- `new()` crée un contrôleur arrêté.
- `start()` active la simulation.
- `stop()` l'arrête.
- `is_running()` lit l'état courant.

SimulationInstance

Une instance correspond à une simulation autonome avec sa carte, son moteur, sa diffusion d'événements et son jeton d'accès.

Champ	Rôle
<code>token</code>	Jeton attendu par le WebSocket.
<code>engine</code>	Moteur actif manipulé par le client et le worker.
<code>initial_engine</code>	Instantané initial réutilisé pour les calculs de score ou les requêtes globales.
<code>broadcast</code>	Canal de diffusion des paquets serveur vers tous les clients connectés.
<code>controller</code>	État marche/arrêt partagé avec le WebSocket.
<code>active_connections</code>	Nombre de clients WebSocket actuellement connectés.
<code>speed_multiplier</code>	Accélération de simulation appliquée par le worker.
<code>file_uuid</code>	Identifiant du fichier de carte persisté, si la simulation vient d'un fichier.

Fonctions de `SimulationInstance`

- `new(map)` construit une instance complète, initialise les véhicules, recalcule les chemins et lance un worker asynchrone qui publie les mises à jour.
- `from_file(path, uuid)` charge une carte persistée puis en crée une instance.
- `new_default()` essaie de charger `data/lannion.osm.pbf` ; si l'import échoue, la simulation démarre sur une carte vide.

Le worker interne publie périodiquement les véhicules, les feux et le paquet de fin de simulation lorsque tout le monde est arrivé ou lorsque la durée maximale est atteinte.

AppState

État partagé du serveur. Il stocke la table des simulations actives, indexées par `uuid`.

generate_token

Construit un jeton hexadécimal de 32 caractères à l'aide d'un générateur aléatoire.

Flux de démarrage

1. `run()` crée l'état partagé vide.
2. `run()` configure les origines autorisées via `ALLOWED_ORIGINS`.
3. `run()` enregistre les routes HTTP et WebSocket.
4. `run()` lie le serveur sur `0.0.0.0:8080`.

Ce qu'il faut retenir

- `handlers.rs` décide des routes et de leurs validations.
- `runner.rs` porte la vie d'une simulation individuelle.
- Le WebSocket ne manipule pas directement les fichiers de carte; il agit sur une `SimulationInstance` déjà existante.

Générateur de Cartes et Véhicules

Ce module fournit des fonctions utilitaires pour créer des environnements de simulation complets, que ce soit à partir de données réelles (OSM) ou pour des tests spécifiques.

Fonctions

create_osm_map

Charge une carte depuis un fichier OSM PBF et configure les types d'intersections.

- **Entrées :**
 - `path: P` : Chemin vers le fichier `.osm.pbf`.
- **Action :**
 1. Analyse le fichier PBF via le module `osm_parser`.
 2. Ne conserve que la plus grande composante connexe de la carte pour éviter les fragments isolés.
 3. Assigne automatiquement des rôles aux nœuds :
 - `Habitation` : Nœuds terminaux avec uniquement des sorties.
 - `Workplace` : Nœuds terminaux avec uniquement des entrées.
 - `Intersection` : Nœuds avec un degré élevé.
 4. Réduit le nombre d'habitations et de lieux de travail pour une distribution plus réaliste (environ 1/3 conservés).
 5. Appelle `build_intersections` pour générer la logique interne des carrefours.
- **Retour :** `Result<Map, OsmParseError>`.

create_random_vehicles

Génère une liste de véhicules avec des trajets aléatoires mais valides.

- **Entrées :**
 - `map: &Map` : La carte de simulation.
 - `count: usize` : Le nombre de véhicules à créer.
- **Action :**
 1. Identifie tous les nœuds de type `Habitation` et `Workplace`.
 2. Utilise un parcours en largeur (BFS) pour identifier les paires (départ, arrivée) qui sont réellement connectées.
 3. Crée des véhicules avec des spécifications variées (Voitures, Bus) et des types de moteurs différents.

4. Répartit les départs sur 24h en utilisant des **plans de trajets pendulaires** (**CommutePlan**) pour simuler un cycle de vie complet.

- **Retour** : `Vec<Vehicle>` .

create_connected_map

Génère une carte aléatoire structurée sous forme de grille ou de graphe connecté.

- **Entrées** :
 - `num_nodes` : Nombre de nœuds souhaités.
 - `width, height` : Dimensions de la zone.
- **Action** : Place les nœuds aléatoirement et tente de créer des connexions pour former un réseau routier cohérent.
- **Retour** : `Map` .

Cartes de Test (Scénarios)

Le module contient plusieurs fonctions créant des micro-cartes pour tester des comportements spécifiques :

- **create_one_intersection_congestion_map** : Une seule intersection avec beaucoup de trafic pour tester les bouchons.
- **create_intersection_test_map** : Une croix standard (4 bras) pour tester les priorités à droite.
- **create_traffic_light_test_map** : Une intersection équipée de feux de signalisation.
- **create_roundabout_test_map** : Un rond-point fonctionnel.
- **create_multilane_test_map** : Routes à plusieurs voies pour tester les changements de file et les dépassements.

link_ids_for_arm

Fonction utilitaire pour récupérer les identifiants de liens entre deux nœuds d'une intersection.

- **Entrées** : `map, from_id, to_id` .
- **Retour** : `Vec<u32>` (liste des IDs de liens).

WebSocket protocol

Fichier principal: `src/api/websocket.rs`. Ce chapitre décrit le protocole JSON échangé sur `/ws`, les structures de données sérialisées par le serveur et la logique interne qui relie le socket à une `SimulationInstance`.

Connexion

ConnectParams

Paramètres de requête attendus lors de l'ouverture du WebSocket.

- `uuid` : identifiant de la simulation à rejoindre.
- `token` : jeton d'authentification associé à cette simulation. Si l'un des deux est invalide, la connexion est immédiatement rejetée avec une fermeture `Unauthorized`.

Paquets clients

ClientPacket

Le client envoie une seule étiquette JSON `id` et un objet `data`. Les variantes actuelles sont les suivantes:

Variante	Rôle
<code>StartSimulation</code>	Démarre l'exécution du moteur.
<code>StopSimulation</code>	Arrête l'exécution du moteur.
<code>ResetSimulation</code>	Réinitialise l'état courant de la simulation.
<code>AddNode</code>	Ajoute un noeud de carte (<code>x</code> , <code>y</code> , <code>kind</code>).
<code>DeleteNode</code>	Supprime un noeud (<code>id</code>).
<code>UpdateNode</code>	Change le type d'un noeud (<code>id</code> , <code>kind</code>).
<code>AddRoad</code>	Ajoute une route orientée (<code>from_id</code> , <code>to_id</code> , <code>lane_count</code> , <code>speed_limit</code>).
<code>DeleteRoad</code>	Supprime une route (<code>id</code>).
<code>UpdateRoad</code>	Modifie la limite de vitesse (<code>id</code> , <code>speed_limit</code>), et éventuellement le nombre de voies (<code>lane_count</code>).

Variante	Rôle
SetSpeed	Modifie la vitesse d'exécution du worker (<code>multiplier</code>).
RequestScore	Demande un score complet.
RequestDensity	Demande une carte de densité par route.
AddWaypoints	Ajoute des points de passage à un véhicule (<code>vehicle_id</code> , <code>node_ids</code>).
RequestVehicles	Demande la liste détaillée des véhicules.
CreateBusLine	Crée une ligne de bus (<code>name</code> , <code>stop_node_ids</code>).
DeleteBusLine	Supprime une ligne de bus (<code>bus_line_id</code>).
RequestBusLines	Demande la liste des lignes de bus.

Le message `UpdateRoad` accepte aussi un `lane_count` optionnel dans la version actuelle du serveur.

Paquets serveur

ServerPacket

| Variante | Rôle | | `Map` | Instantané complet de la carte. | | `VehicleUpdate` | État temps réel des véhicules et des feux. | | `MapEdit` | Réponse à une opération d'édition de carte. | | `Score` | Résultat complet de la simulation. | | `SimulationFinished` | Notification de fin de simulation. | | `DensityMap` | Résultat de la requête de densité par route. | | `VehicleList` | Liste détaillée des véhicules. | | `BusLineList` | Liste des lignes de bus. |

Fonctions de contrôle du socket

ws_handler

Valide l'accès et initialise la connexion WebSocket.

- **Entrées** : `ws: WebSocketUpgrade` , `params: ConnectParams` , `state: Arc<AppState>` .
- **Action** : Vérifie la validité du `uuid` de simulation et du `token` . Si valide, passe le socket à `ws_loop` .
- **Retour** : `Response` .

ws_loop

Gère la communication bidirectionnelle continue.

- **Entrées** : `socket: WebSocket`, `instance: Arc<SimulationInstance>`, `state: Arc<AppState>`, `uuid: Uuid`.
- **Action** :
 1. Envoie l'état initial de la carte au client.
 2. Gère en parallèle les messages entrants du client (via `handle_client_packet`) et les messages sortants du worker (broadcast).
 3. Gère la déconnexion et le nettoyage de l'instance si plus aucun client n'est connecté.

process_incoming_msg

Traite un message brut reçu depuis le socket.

- **Action** : Décode le JSON en `ClientPacket` et appelle `handle_client_packet`.
- **Retour** : `bool` (continuer la boucle).

process_broadcast_msg

Relaye un message interne du serveur vers le client.

- **Action** : Sérialise le `ServerPacket` et l'envoie sur le socket.
- **Retour** : `bool`.

handle_client_packet

Point d'entrée de la logique métier du WebSocket.

- **Entrées** : `packet: ClientPacket`, `socket`, `instance`.
- **Action** : Distribue l'action vers les fonctions appropriées (moteur ou éditeur) en vérifiant les droits (ex: interdiction d'éditer si la simulation tourne).

send_edit_error

Envoie un paquet `MapEdit` signalant un échec au client.

broadcast_map_edit_success

Diffuse un paquet `MapEdit` signalant un succès à tous les clients connectés.

serialize_map

Transforme la structure `Map` en données JSON pour le frontend.

- **Entrées** : `map: &Map` .
- **Action** : Parcourt le graphe pour extraire les positions des nœuds, leurs types, et les caractéristiques des routes (voies, longueur, vitesse).
- **Retour** : `(Vec<Value>, Vec<Value>)` (Liste des nœuds, Liste des arêtes).

serialize_vehicle

Produit la représentation détaillée d'un véhicule pour le flux temps réel. Le format inclut l'identifiant, la position, l'orientation, le type logique, l'état, la motorisation, l'origine, la destination et les waypoints.

serialize_vehicle_summary

Produit une version condensée d'un véhicule, utilisée pour les listes et les réponses plus légères.

serialize_bus_line

Produit la sérialisation d'une ligne de bus avec son identifiant, son nom, ses arrêts et le véhicule associé.

serialize_intersection_kind

Convertit une chaîne reçue du client en `IntersectionKind` . Cette fonction centralise la validation des chaînes comme `Habitation` , `Intersection` et `Workplace` .

serialize_traffic_lights

Construit la vue JSON des contrôleurs de feux actifs. Pour chaque contrôleur, le serveur liste les routes actuellement vertes sous forme de `green_road_ids` .

Formes de données importantes

Carte

Le paquet `Map` contient deux tableaux:

- `nodes` : noeuds du graphe avec position, métadonnées et liste des `internal_lanes` (trajectoires d'intersection);
- `edges` : routes avec longueur, vitesse, nombre de voies et largeur de voie.

Véhicule

Le paquet `VehicleUpdate` contient:

- `vehicles` : positions et états de tous les véhicules;
- `traffic_lights` : ensemble des feux et de leurs routes ouvertes.

Score

Le paquet `Score` expose les mesures finales:

- `score` : Valeur finale sur 100.
- `total_trip_time / ref_total_trip_time` : Temps cumulé réel vs théorique.
- `total_emitted_co2 / ref_total_emitted_co2` : CO2 cumulé réel vs théorique.
- `network_length / ref_network_length` : Longueur d'infrastructure réelle vs borne inférieure de Steiner.
- `success_rate` : Ratio de véhicules arrivés à destination.

Ce qu'il faut retenir

- Le WebSocket est le canal de temps réel de la simulation.
- L'édition de carte est bloquée tant que la simulation tourne.
- Le serveur distingue toujours entre les réponses légères (`VehicleList` , `BusLineList`) et les instantanés complets (`Map` , `MapEdit`).

Exemples JSON

Client -> Serveur (Démarrer simulation)

```
{ "id": "StartSimulation", "data": {} }
```

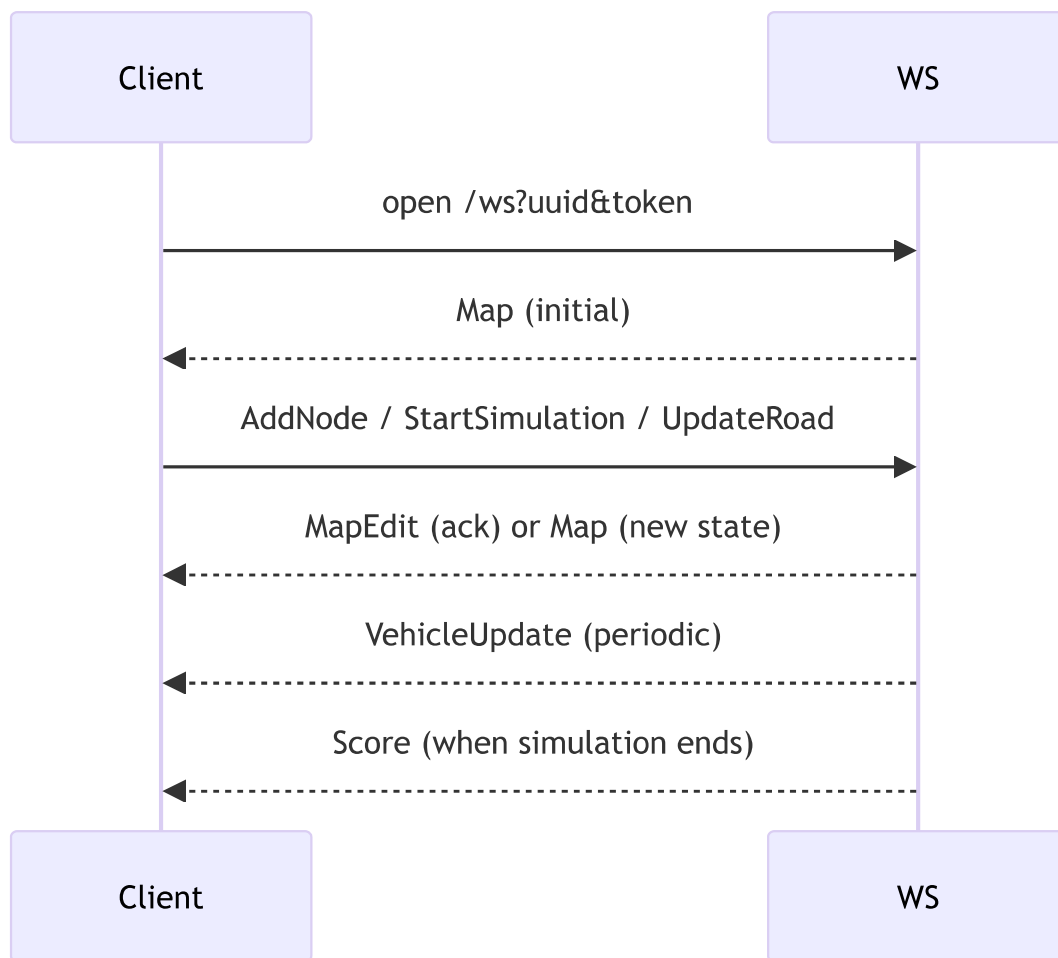
Client -> Serveur (Ajouter noeud)

```
{ "id": "AddNode", "data": { "x": 123.4, "y": 56.7, "kind": "Intersection" } }
```

Serveur -> Client (Mise à jour véhicules)

```
{  
  "id": "VehicleUpdate",  
  "data": {  
    "vehicles": [ { "id": 1001, "x": 12.3, "y": 45.6, "heading": 1.57,  
"kind": "Car", "state": "Moving" } ],  
    "traffic_lights": [ { "id": 5, "green_road_ids": [2, 3] } ]  
  }  
}
```

Runtime sequence (WebSocket)



WebSocket Examples

Ce fichier rassemble des exemples JSON envoyés et reçus via le WebSocket.

Format général

Les paquets utilisent `serde` avec `tag = "id"` et `content = "data"`, `rename_all = "camelCase"`.

Exemple générique:

```
{ "id": "AddNode", "data": { "x": 12.3, "y": 45.6, "kind": "Intersection" } }
```

Client → Serveur (ClientPacket) exemples

- Start / Stop / Reset:

```
{ "id": "StartSimulation", "data": {} }  
{ "id": "StopSimulation", "data": {} }  
{ "id": "ResetSimulation", "data": {} }
```

- Edition de nœud:

```
{ "id": "AddNode", "data": { "x": 10.0, "y": 5.0, "kind": "Intersection" } }  
{ "id": "DeleteNode", "data": { "id": 17 } }  
{ "id": "MoveNode", "data": { "id": 17, "x": 12.0, "y": 6.0 } }  
{ "id": "UpdateNode", "data": { "id": 17, "kind": "Workplace" } }
```

- Edition de route:

```
{ "id": "AddRoad", "data": { "fromId": 3, "toId": 4, "laneCount": 2,  
"speedLimit": 13.9 } }  
{ "id": "DeleteRoad", "data": { "id": 21 } }  
{ "id": "UpdateRoad", "data": { "id": 21, "speedLimit": 11.1 } }
```

Note: les clés internes suivent `camelCase` à la sérialisation (par ex. `laneCount`, `speedLimit`).

Serveur → Client (ServerPacket) exemples

- Map (extrait de `serialize_map`):

```
{
  "id": "Map",
  "data": {
    "nodes": [
      { "id": 1, "kind": "Intersection", "x": 10.0, "y": 5.0,
"has_traffic_light": false, "radius": 4.0 }
    ],
    "edges": [
      { "id": 2, "from": 1, "to": 3, "lane_count": 2, "lane_width": 3.0,
"length": 120.0, "speed_limit": 13.9 }
    ]
  }
}
```

- VehicleUpdate:

```
{
  "id": "VehicleUpdate",
  "data": {
    "vehicles": [
      {
        "id": 5,
        "x": 11.2,
        "y": 6.3,
        "heading": 1.5708,
        "kind": "Car",
        "state": "Moving"
      }
    ],
    "traffic_lights": [ { "id": 1, "green_road_ids": [2] } ]
  }
}
```

- MapEdit (success / failure):

```
{
  "id": "MapEdit",
  "data": {
    "success": true,
    "error": null,
    "nodes": [ { "id": 1, "kind": "Habitation", "x": 0, "y": 0,
"has_traffic_light": false, "radius": 3.0 } ],
    "edges": [ { "id": 2, "from": 1, "to": 3, "lane_count": 2, "lane_width":
3.0, "length": 120.0, "speed_limit": 13.9 } ]
  }
}

{
  "id": "MapEdit",
  "data": { "success": false, "error": "Stop simulation before editing the
map", "nodes": [], "edges": [] }
}
```

- Score :

```
{
  "id": "Score",
  "data": {
    "score": 123.4,
    "totalTripTime": 456.7,
    "totalEmittedCo2": 12.3,
    "networkLength": 789.0,
    "totalDistanceTraveled": 345.6,
    "successRate": 0.92
  }
}
```

Usage: copiez ces exemples dans la console WebSocket de votre client (ou dans l'outil
WebSocket du navigateur) pour simuler interactions.

Map module

Le sous-module `map` contient toute la géométrie et toute la topologie routière utilisées par le simulateur.

Pages de référence:

- [Map model](#) : structure centrale de la carte, réglages et persistance.
- [Roads & Lanes](#) : routes, voies, liens de traversée et relations de conflit.
- [Intersections](#) : génération des mouvements internes et arbitrage des conflits.
- [Map editor](#) : fonctions de modification de la topologie.
- [OSM parser](#) : import d'un graphe routier depuis un fichier `.osm.pbf`.
- [Traffic lights](#) : contrôleurs de feux et phases.
- [Roundabouts](#) : handle de création et finalisation des ronds-points.

Pages d'aide:

- [Link IDs & how to obtain them](#)
- [Scenarios \(deadlock & priority\)](#)
- [Editor examples](#)

Map model

Fichier: `src/map/model.rs`.

Ce module est la source de vérité de la topologie routière. Il contient la carte complète, ses réglages globaux, les identifiants utilisés par le moteur et les helpers de persistance.

MapSettings

Les réglages de la carte contrôlent à la fois la génération des véhicules, le calcul du score et les paramètres de budget du projet.

Champ	Rôle	Valeur par défaut
<code>vehicle_count</code>	Nombre de véhicules générés au départ.	500
<code>simulation_duration</code>	Durée maximale de la simulation.	600.0 s
<code>max_budget</code>	Budget de référence pour l'infrastructure.	750000000
<code>base_cost_per_meter</code>	Coût d'une route par mètre.	500
<code>intersection_cost</code>	Coût d'une intersection.	50000
<code>habitation_cost</code>	Coût associé aux zones d'habitation.	150000
<code>workplace_cost</code>	Coût associé aux zones de travail.	200000
<code>time_weight</code>	Poids de la dimension temps dans le score.	0.4
<code>success_weight</code>	Poids du taux de réussite.	0.2
<code>pollution_weight</code>	Poids de la pollution.	0.2
<code>infrastructure_weight</code>	Poids de l'infrastructure.	0.2
<code>use_day_night_cycle</code>	Active ou désactive le cycle jour/nuit.	<code>true</code>

SavedBusLine

Ligne de bus persistée sur disque.

- `id` : identifiant de ligne.

- `name` : nom lisible.
- `stop_node_ids` : identifiants des arrêts.

Map

La carte contient:

- un graphe orienté `Intersection -> Road` ;
- une table de correspondance entre identifiants publics et indices `petgraph` ;
- des compteurs d'identifiants pour les noeuds, routes, liens et contrôleurs;
- les contrôleurs de feux;
- le nom de la carte et ses réglages;
- les lignes de bus sauvegardées.

Champs principaux

Champ	Rôle
<code>graph</code>	Graphe routier principal.
<code>node_index_map</code>	Correspondance <code>id -> NodeIndex</code> .
<code>next_node_id</code>	Prochain identifiant de noeud à attribuer.
<code>next_edge_id</code>	Prochain identifiant de route à attribuer.
<code>next_link_id</code>	Prochain identifiant de lien interne.
<code>next_controller_id</code>	Prochain identifiant de contrôleur de feux.
<code>traffic_lights</code>	Contrôleurs de feux actifs.
<code>name</code>	Nom lisible de la carte.
<code>settings</code>	Réglages globaux.
<code>bus_lines</code>	Lignes de bus persistées.
<code>next_bus_line_id</code>	Prochain identifiant de ligne de bus.

Méthodes utiles

add_intersection

Ajoute un nœud à la carte.

- **Entrées :**
 - `kind: IntersectionKind` : Le type de nœud (Habitation, Workplace, etc.).
 - `x, y: f32` : Les coordonnées géographiques projetées.

- **Action** : Insère un nouveau nœud dans le graphe et met à jour la table de correspondance des IDs.
- **Retour** : `u32` (l'identifiant public du nœud).

add_road

Ajoute une route orientée entre deux intersections.

- **Entrées** :
 - `from, to: u32` : Identifiants publics des nœuds de départ et d'arrivée.
 - `lane_count: u8` : Nombre de voies.
 - `speed_limit: f32` : Vitesse limite en m/s.
 - `length: f32` : Longueur en mètres.
- **Action** : Crée une arête dans le graphe reliant les deux nœuds.
- **Pré-conditions** : Les nœuds `from` et `to` doivent exister.
- **Retour** : `u32` (l'identifiant public de la route).

add_two_way_road

Ajoute une liaison bidirectionnelle.

- **Action** : Appelle `add_road` deux fois avec les directions inversées.
- **Entrées** : `from, to: u32, lane_count: u8, speed_limit: f32, length: f32`.
- **Retour** : `(u32, u32)` (les identifiants des deux routes créées).

find_node / find_edge

Convertit un identifiant public (`u32`) en un index interne (`NodeIndex` OU `EdgeIndex`).

- **Entrées** : `id: u32`.
- **Retour** : `Option<NodeIndex>` OU `Option<EdgeIndex>`.

neighboring_intersections

Liste les nœuds voisins accessibles directement depuis un nœud donné.

- **Entrées** : `source: NodeIndex`.
- **Retour** : `Vec<NodeIndex>`.

intersection_neighbor_distance

Donne la longueur de la route reliant deux intersections adjacentes.

- **Entrées** : `source, destination`.

- **Retour** : `Option<f32>` .

intersections_euclidean_distance

Calcule la distance à vol d'oiseau entre deux intersections.

- **Entrées** : `source` , `destination` .
- **Retour** : `f32` (mètres).

retain_largest_component

Nettoie la carte pour ne garder que la partie principale.

- **Action** : Identifie les composantes faiblement connexes du graphe (en ignorant l'orientation des routes) et supprime tous les nœuds et arêtes n'appartenant pas à la plus grande composante.
- **Post-conditions** : Garantit que le réseau routier est d'un seul bloc, évitant les véhicules bloqués sur des fragments isolés.

save / load

Gère la persistance de la carte au format JSON.

- **Entrées** : `path`: `P` (chemin du fichier).
- **Retour** : `Result` .

Coordinates

Structure légère utilisée pour les positions dans le monde simulé.

- `x` : coordonnée horizontale.
- `y` : coordonnée verticale.

Lecture fonctionnelle

- `Map` décrit la topologie.
- `MapSettings` décrit comment cette topologie doit être utilisée dans la simulation.
- `SavedBusLine` permet de persister les transports publics sans les confondre avec la structure routière.
- `Coordinates` est le type élémentaire réutilisé partout où une position est nécessaire.

Roads & Lanes

Fichier: `src/map/road.rs`.

Ce module contient les structures qui décrivent une route, ses voies et les mouvements possibles à travers les intersections.

LinkType

Type sémantique d'un mouvement à travers une intersection.

- `Yield` : cédez le passage.
- `Priority` : mouvement prioritaire.
- `Stop` : arrêt obligatoire.
- `TrafficLight` : mouvement contrôlé par un feu.

Road

Section orientée du graphe entre deux intersections.

Champ	Rôle
<code>id</code>	Identifiant unique de la route.
<code>length</code>	Longueur utile de la section.
<code>speed_limit</code>	Limite de vitesse de la route.
<code>lane_width</code>	Largeur nominale d'une voie.
<code>lanes</code>	Voies appartenant à cette route.

Road::new

Construit une route avec plusieurs voies.

- **Entrées :**
 - `id` : Identifiant public de la route.
 - `lane_count` : Nombre de voies à créer.
 - `speed_limit` : Vitesse limite (m/s).
 - `length` : Longueur de la route (m).
- **Action :** Initialise la structure `Road` et crée le nombre demandé d'objets `Lane`. La vitesse est bornée par `MAX_SPEED`.

- **Retour** : Une instance de `Road` .

Lane

Voie de circulation appartenant à une route.

Champ	Rôle
<code>id</code>	Identifiant local de la voie.
<code>road_id</code>	Identifiant de la route parente.
<code>length</code>	Longueur de la voie.
<code>speed_limit</code>	Limite de vitesse copiée depuis la route.
<code>links</code>	Mouvements autorisés depuis cette voie.

Link

Mouvement logique reliant une voie entrante à une route sortante via une intersection.

Champ	Rôle
<code>id</code>	Identifiant global du mouvement.
<code>lane_origin_id</code>	Identifiant local de la voie entrante.
<code>lane_destination_id</code>	Identifiant local de la voie de sortie.
<code>via_internal_lane_id</code>	Identifiant de la voie interne traversée.
<code>destination_road_id</code>	Route de sortie visée.
<code>link_type</code>	Règle de priorité appliquée.
<code>entry</code>	Point géométrique d'entrée dans le carrefour.
<code>junction_center</code>	Centre géométrique du carrefour.
<code>foe_links</code>	Mouvements concurrents pouvant bloquer ce lien.
<code>foe_internal_lane_ids</code>	Identifiants des voies internes en conflit direct.

FoeLink

Mouvement adverse associé à un lien en conflit.

Champ	Rôle
<code>id</code>	Identifiant du lien adverse.

Champ	Rôle
<code>link_type</code>	Type de priorité de ce lien adverse.
<code>entry</code>	Point d'entrée géométrique du mouvement adverse.

Lecture fonctionnelle

- `Road` porte la structure de haut niveau.
- `Lane` porte l'état de circulation local sur la route.
- `Link` porte les mouvements possibles dans les intersections.
- `FoeLink` porte la notion de conflit.

Rôle et utilisation

- Les `links` d'une `Lane` correspondent aux mouvements autorisés à la traversée d'une intersection. Ils sont référencés par `DrivePlanEntry.link_id` lors de la planification et servent de clef pour `link_states` (enregistrement des approches). `foe_links` représente conflits physiques (croisements / merges).

JSON / payload (format côté API)

Exemple `Road` JSON (tel que sérialisé par `serialize_map`):

```
{  
  "id": 2,  
  "from": 1,  
  "to": 3,  
  "lane_count": 2,  
  "lane_width": 3.0,  
  "length": 120.0,  
  "speed_limit": 13.9  
}
```

Exemple conceptuel d'un `Link` (non directement sérialisé entier par le serveur, mais utilisé par l'engine):

```
{  
  "id": 101,  
  "destination_road_id": 3,  
  "via_internal_lane_id": 55,  
  "link_type": "Yield",  
  "foe_links": [ { "id": 201, "link_type": "Priority" } ]  
}
```

Notes:

- Les `link.id` sont uniques et globalement utilisés pour l'enregistrement d'approches et la synchronisation entre clients et engine.
- Les structures internes (Lane, Link, FoeLink) servent principalement à la logique de planification; l'API n'expose que un sous-ensemble via `serialize_map`.

Intersections

Fichier: `src/map/intersection.rs`.

Les intersections transforment deux routes abstraites en une géométrie de traversée, puis en règles de priorité et de conflit exploitables par le moteur.

Types de base

IntersectionKind

Type fonctionnel du noeud.

- `Habitation` : zone d'origine potentielle pour les véhicules.
- `Intersection` : carrefour standard.
- `Workplace` : zone de destination potentielle.
- `TrafficLight` : intersection contrôlée par des feux tricolores.

Intersection

Noeud du graphe routier.

Champ	Rôle
<code>id</code>	Identifiant public du noeud.
<code>kind</code>	Rôle fonctionnel du noeud.
<code>center_coordinates</code>	Position du carrefour.
<code>radius</code>	Rayon utilisé pour la géométrie d'entrée/sortie.
<code>internal_lanes</code>	Trajectoires internes construites par <code>build_intersections</code> .

InternalLane

Trajectoire à l'intérieur du carrefour.

Champ	Rôle
<code>id</code>	Identifiant interne.
<code>from_lane_id</code>	Voie d'entrée.
<code>to_lane_id</code>	Voie de sortie.

Champ	Rôle
length	Longueur géométrique du passage.
speed_limit	Limite de vitesse pour ce passage.
entry	Point d'entrée géométrique.
exit	Point de sortie géométrique.

ApproachData

Fenêtre temporelle réservée par un véhicule sur un lien.

Champ	Rôle
arrival_time	Heure prévue d'entrée.
leave_time	Heure prévue de sortie.
arrival_speed	Vitesse à l'entrée.
leave_speed	Vitesse à la sortie.
will_pass	Indique si le véhicule devrait effectivement franchir le carrefour.

LinkState

État dynamique associé à un lien, surtout pour savoir quels véhicules l'ont réservé.

- `approaching : table vehicle_id -> ApproachData .`

Construction des mouvements

build_intersections(map)

Recalcule toute la topologie d'intersection à partir du graphe actuel.

- **Entrée** : `map : &mut Map .`
- **Action** :
 1. Réinitialise les identifiants de liens globaux de la carte.
 2. Pour chaque intersection, vide les trajectoires internes (`internal_lanes`).
 3. Pour chaque route, vide la liste des liens sortants des voies.
 4. Appelle `build_intersection` pour chaque nœud du graphe.
- **Post-conditions** : La carte est prête pour la simulation avec des intersections configurées géométriquement et logiquement.

build_intersection(map, junction)

Construit les mouvements et détecte les conflits pour un carrefour unique.

- **Entrées :**
 - `map`: &mut Map .
 - `junction`: NodeIndex .
- **Action :**
 1. Identifie toutes les routes entrantes et sortantes.
 2. Génère toutes les paires possibles (route_entrante, route_sortante) si elles ne font pas demi-tour (U-turn).
 3. **RawLink** : Utilise une structure interne temporaire pour stocker les caractéristiques de chaque liaison candidate :
 - Arêtes source et destination.
 - Indices de voies.
 - Coordonnées de l'entrée et de la sortie sur le bord du carrefour.
 4. Crée les `InternalLane` et les `Link` (liens logiques).
 5. Analyse les intersections géométriques entre les trajectoires pour peupler les listes de conflits (`foe_links`).
 6. Assigne les priorités (Stop, Yield, Priority) selon la configuration. L'utilisateur peut modifier ce type via l'éditeur.

Aides géométriques

node_coords

Récupère les coordonnées d'un nœud.

- **Retour** : (f32, f32) .

boundary_point

Calcule un point à la périphérie de l'intersection (en fonction de son rayon) dans la direction d'un voisin.

- **Entrées** : centre (jx, jy), rayon, point cible (px, py).
- **Retour** : (f32, f32) .

segments_intersect

Détermine si deux segments de droite $(p1, p2)$ et $(p3, p4)$ se croisent physiquement. Utilisé pour la détection de conflits de trajectoires.

- **Retour** : `bool` .

lane_boundary_point

Décale un point de bordure perpendiculairement à la direction de la route pour aligner une voie spécifique.

- **Entrées** : point de base, vecteur perpendiculaire, index de la voie, largeur de voie.
- **Retour** : $(f32, f32)$.

dist

Calcule la distance Euclidienne entre deux points.

- **Retour** : `f32` .

cross

Calcule le produit vectoriel de deux vecteurs (OA et OB). Utilisé pour tester l'orientation de points.

- **Retour** : `f32` .

on_segment

Vérifie si un point q se trouve sur le segment pr .

- **Retour** : `bool` .

lerp

Interpolation linéaire entre deux valeurs a et b .

- **Entrées** : a, b, t (facteur entre 0 et 1).
- **Retour** : `f32` .

perp_right

Calcule le vecteur perpendiculaire à droite d'un vecteur directionnel donné.

- **Entrées** : `dx` , `dy` .
- **Retour** : `(f32, f32)` .

Arbitrage et priorité

foe_is_to_the_right

Détermine si un lien adverse se trouve à la droite du lien courant (priorité à droite).

- **Retour** : `bool` .

time_window_conflict

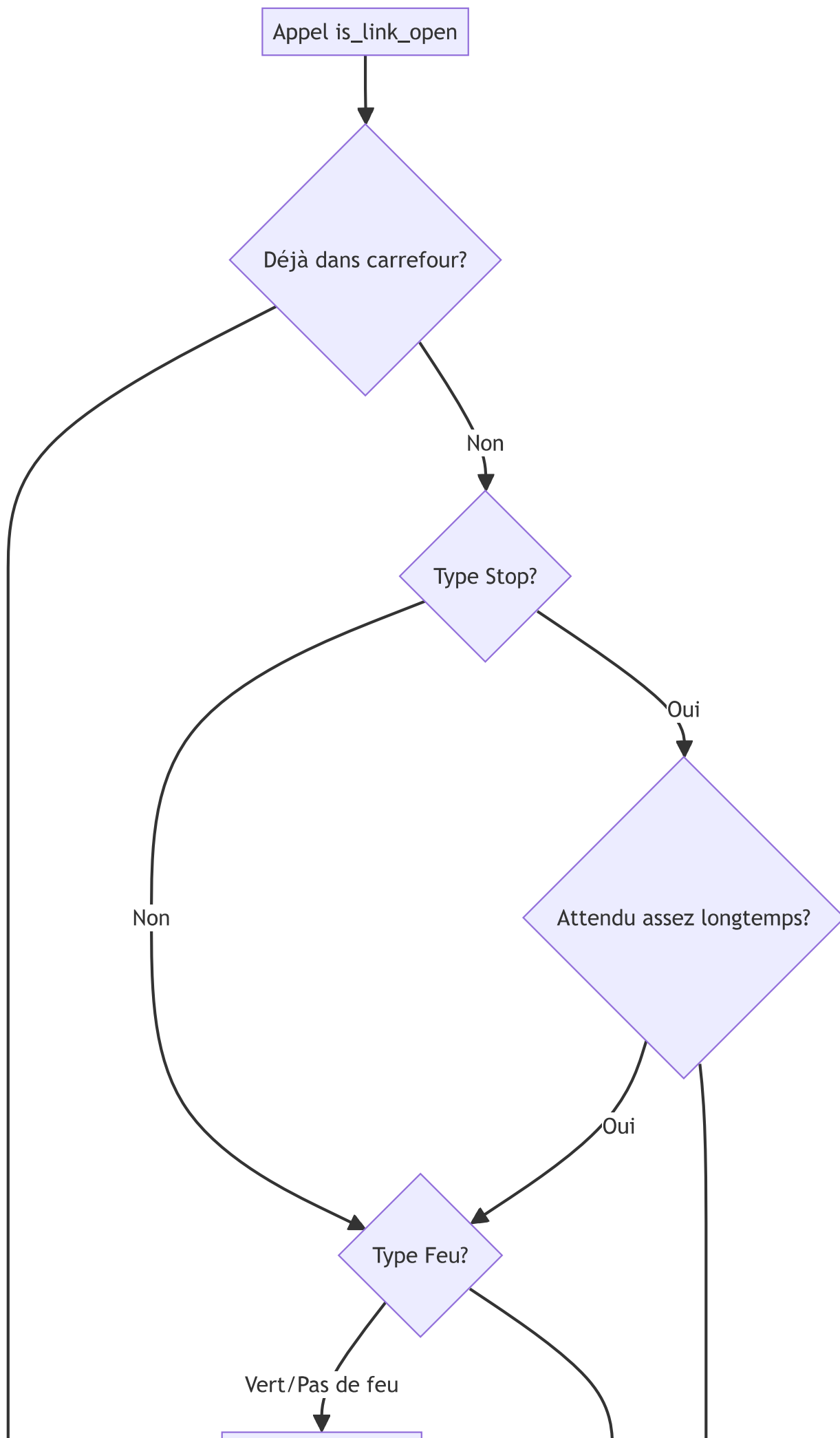
Vérifie si deux fenêtres d'occupation d'une intersection se chevauchent.

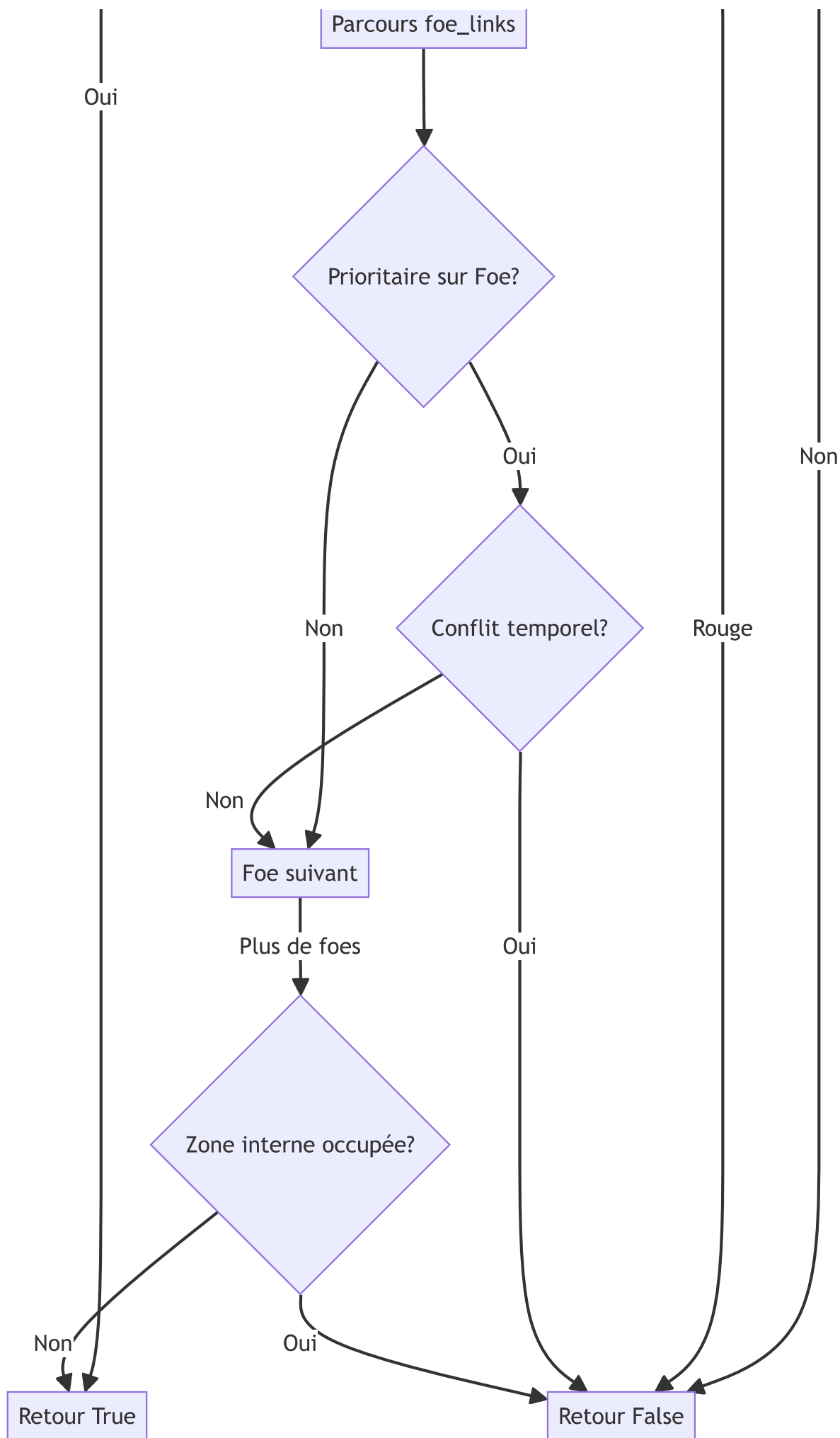
- **Entrées** : heures d'arrivée et de sortie des deux véhicules, vitesses, impatience.
- **Action** : Compare les intervalles temporels en tenant compte des marges de sécurité.
- **Retour** : `bool` .

is_link_open

Vérifie si un véhicule peut s'engager sur un lien à un instant donné.

- **Entrées** : `link` , `vehicle` , `ego_data` , `link_states` , `vehicles_by_lane` , `vehicles` , `junction_id` , `look_ahead` , `stop_dwell_time` , `green_links` .
- **Action** :
 - Vérifie si le lien est contrôlé par un feu et si celui-ci est vert.
 - Applique l'arrêt obligatoire (dwell time) pour les panneaux Stop.
 - Pour chaque lien adverse en conflit (`foe_link`) :
 - Vérifie si un véhicule plus prioritaire a déjà réservé une fenêtre temporelle conflictuelle.
 - Applique la règle de priorité à droite si les deux véhicules ont la même priorité de base.
 - Vérifie si la zone de conflit interne est déjà occupée physiquement.
 - Gère l'impatience : un véhicule bloqué trop longtemps réduit ses marges de sécurité perçues.
- **Retour** : `bool` .





Ce qu'il faut retenir

- `build_intersections` est appelée après toute modification de la topologie.
- Les conflits sont résolus à partir de la géométrie et des fenêtres temporelles.
- `ApproachData` et `LinkState` sont les objets centraux de l'arbitrage.

Map editor

Fichier principal: `src/map/editor.rs`.

Ce chapitre documente les fonctions de modification de la carte utilisées par l'interface HTTP et le WebSocket.

Fonctions

add_node

Ajoute un noeud au graphe avec les coordonnées fournies et un `IntersectionKind`. La fonction renvoie l'identifiant public du noeud créé.

delete_node

Supprime un noeud par son identifiant, met à jour la table des indices et reconstruit les intersections. Si le noeud est absent, la fonction renvoie une erreur textuelle.

update_node

Change uniquement le type fonctionnel d'un noeud. La topologie routière n'est pas reconstruite automatiquement par cette fonction; elle doit être recalculée si le changement de rôle a un impact sur les trajets.

add_road

Crée une route orientée entre deux noeuds existants. La longueur utile est calculée à partir de la distance géométrique entre les centres, en retirant les rayons des intersections pour obtenir une longueur exploitable par les véhicules. La topologie d'intersection est ensuite reconstruite.

delete_road

Supprime une route existante par son identifiant, puis reconstruit les intersections pour supprimer les liens devenus invalides.

update_road

Met à jour la vitesse limite d'une route et, si demandé, son nombre de voies. Les voies sont recrées proprement afin de repartir d'un état cohérent, puis la topologie d'intersection est recalculée.

add_roundabout

Construit un rond-point en créant un anneau de noeuds et de routes autour d'un centre géométrique. Le résultat est encapsulé dans un `RoundaboutHandle` qui contient les identifiants des noeuds et des routes de l'anneau.

Contraintes importantes:

- le nombre d'armes doit être au moins 3;
- le rayon doit être strictement positif;
- le nombre de voies de l'anneau doit être au moins 1;
- le rayon doit être assez grand pour laisser la place physique aux bras du rond-point.

add_traffic_light_controller

Crée un contrôleur de feux sur une intersection donnée.

Les phases sont fournies comme une liste de triplets: identifiants de liens verts, durée du vert, durée du jaune. La fonction marque les liens concernés comme `TrafficLight` et ajoute le contrôleur dans la carte.

Ce qu'il faut retenir

- Les fonctions de ce module sont celles qui modifient réellement la topologie.
- Elles doivent être suivies d'une reconstruction des intersections dès qu'elles changent la connectivité ou les priorités.
- Les handles retournés par les helpers complexes servent à finaliser correctement la topologie après coup.

Analyseur OSM

Le module `osm_parser` est responsable de la lecture des fichiers de données OpenStreetMap (au format PBF) et de leur conversion en une structure de carte (`Map`) utilisable par le moteur de simulation.

Énumérations

OsmParseError

Représente les erreurs possibles lors de l'analyse d'un fichier OSM.

- **Variantes :**
 - `Io(std::io::Error)` : Erreur d'entrée/sortie lors de l'ouverture ou de la lecture du fichier.
 - `Osm(osmpbf::Error)` : Erreur spécifique à la bibliothèque `osmpbf` lors du décodage des données.
 - `NoHighways` : Lancée si aucune route circulaire (highway) n'est trouvée dans le fichier fourni.

Fonctions

parse_osm_pbf

Point d'entrée principal pour charger une carte depuis un fichier PBF.

- **Entrées :**
 - `path: P` : Le chemin vers le fichier `.osm.pbf` (doit implémenter `AsRef<Path>`).
- **Action :**
 1. Effectue une première passe pour collecter les routes et compter les références des nœuds.
 2. Identifie les intersections (nœuds référencés par plusieurs routes ou extrémités de routes).
 3. Effectue une deuxième passe pour récupérer les coordonnées géographiques (latitude/longitude) des nœuds nécessaires.
 4. Construit la structure `Map` en projetant les coordonnées géographiques dans un plan 2D.

- **Pré-conditions** : Le fichier doit être au format PBF valide.
- **Post-conditions** : Renvoie une `Map` peuplée avec les intersections et les routes si succès.
- **Retour** : `Result<Map, OsmParseError>` .

collect_highway_data

Collecte les données de routes et compte les références des nœuds (Passe 1).

- **Entrées** :
 - `path`: `&Path` : Chemin du fichier.
- **Action** : Parcourt tous les éléments "Way" du fichier OSM, filtre ceux qui correspondent à des types de routes acceptés et non privés, et extrait leurs métadonnées (vitesse, voies, sens unique).
- **Pré-conditions** : Le fichier doit être accessible.
- **Post-conditions** : Aucune modification d'état global.
- **Retour** : `Result<(Vec<HighwayWay>, HashMap<i64, u32>), OsmParseError>` .

collect_node_coords

Récupère les coordonnées des nœuds nécessaires (Passe 2).

- **Entrées** :
 - `path`: `&Path` : Chemin du fichier.
 - `needed`: `&HashSet<i64>` : Ensemble des IDs de nœuds dont on a besoin.
- **Action** : Parcourt les nœuds du fichier OSM et stocke les coordonnées (lat/lon) de ceux présents dans l'ensemble `needed` .
- **Pré-conditions** : L'ensemble `needed` doit contenir les IDs identifiés lors de la passe 1.
- **Post-conditions** : Aucune.
- **Retour** : `Result<HashMap<i64, NodeCoord>, OsmParseError>` .

build_map

Construit l'objet `Map` final.

- **Entrées** :
 - `ways`: `&[HighwayWay]` : Liste des routes collectées.
 - `node_ref_count`: `&HashMap<i64, u32>` : Nombre de fois que chaque nœud est utilisé.
 - `node_coords`: `&HashMap<i64, NodeCoord>` : Coordonnées géographiques des nœuds.
- **Action** :

1. Calcule le centre géographique de la zone pour la projection.
2. Divise les chemins OSM en segments individuels entre chaque intersection.
3. Calcule la longueur réelle de chaque segment.
4. Ajoute les intersections et les routes à l'objet `Map`.

- **Pré-conditions** : Les données fournies doivent être cohérentes.
- **Post-conditions** : Crée une carte centrée sur la zone analysée.
- **Retour** : `Result<Map, OsmParseError>`.

compute_segment_length

Calcule la longueur totale d'une suite de nœuds en mètres.

- **Entrées** :
 - `node_ids: &[i64]` : Liste des IDs de nœuds formant le segment.
 - `coords: &HashMap<i64, NodeCoord>` : Dictionnaire des coordonnées.
- **Action** : Additionne les distances Haversine entre chaque paire consécutive de nœuds.
- **Retour** : `f32` (distance en mètres).

haversine_distance

Calcule la distance à vol d'oiseau entre deux points géographiques.

- **Entrées** : `lat1, lon1, lat2, lon2 (f64)`.
- **Action** : Applique la formule de Haversine pour tenir compte de la courbure de la Terre.
- **Retour** : `f64` (distance en mètres).

project_coords

Projette des coordonnées géographiques sur un plan 2D (équirectangulaire).

- **Entrées** :
 - `lat, lon` : Point à projeter.
 - `center_lat, center_lon` : Point d'origine (0,0) de la projection.
- **Action** : Convertit les degrés en mètres relatifs au centre. L'axe Y est inversé pour correspondre aux systèmes de coordonnées graphiques (Y croissant vers le bas).
- **Retour** : `(f32, f32)` (x, y en mètres).

parse_speed_limit

Analyse la balise `maxspeed` d'OSM.

- **Entrées** : `tag: &str` .
- **Action** : Convertit des chaînes comme “50”, “30 mph”, ou “walk” en une valeur numérique en m/s.
- **Retour** : `Option<f32>` .

default_speed_limit

Fournit une vitesse par défaut selon le type de route.

- **Entrées** : `highway_type: &str` .
- **Action** : Retourne une vitesse standard (ex: 130 km/h pour une autoroute) convertie en m/s.
- **Retour** : `f32` .

Traffic lights

Fichier principal: `src/map/traffic_light.rs`.

Ce chapitre décrit les contrôleurs de feux associés à une intersection et les phases qui règlent les liens autorisés.

Structures

SignalPhase

Une phase décrit l'état d'un feu tricolore pendant une durée donnée. Le moteur alterne entre les phases séquentiellement.

- `green_link_ids` : liste des liens autorisés (ouverts) pendant cette phase;
- `green_duration` : durée pendant laquelle les liens sont au vert (s);
- `yellow_duration` : durée du jaune (période de transition avant la phase suivante) (s).

TrafficLightController

Contrôleur de feux attaché à une intersection.

Champ	Rôle
<code>id</code>	Identifiant du contrôleur.
<code>intersection_id</code>	Identifiant de l'intersection commandée.
<code>phases</code>	Séquence des phases de feu.

TrafficLightControllerHandle

Petit handle renvoyé par les fonctions de création pour référencer un contrôleur récemment créé.

- `controller_id` : identifiant à réutiliser pour la suite.

États et dynamique

TrafficLightRuntimeState

Structure interne utilisée par le moteur de simulation pour suivre l'évolution temporelle de chaque contrôleur.

- `phase_index` : index de la phase actuelle dans le vecteur `phases`.
- `time_in_phase` : temps écoulé depuis le début de la phase actuelle (s).

Cycle de mise à jour

À chaque pas de simulation, le moteur effectue les opérations suivantes :

1. **Incrémentation** : Ajoute `delta_t` au `time_in_phase` de chaque contrôleur.
2. **Transition** : Si le temps dépasse la durée totale de la phase (`green_duration` + `yellow_duration`), le contrôleur passe à la phase suivante (ou revient à la première) et réinitialise son compteur.
3. **Actualisation des liens** : La liste globale `green_links` est vidée puis remplie avec les identifiants de liens (`green_link_ids`) de toutes les phases actuellement au vert (celles dont le `time_in_phase` est inférieur à `green_duration`).

Note sur le jaune : Pendant la `yellow_duration` , aucun lien n'est ajouté à `green_links` pour ce contrôleur, ce qui interdit tout nouvel engagement dans le carrefour.

Lecture fonctionnelle

- les phases sont définies en termes de liens internes autorisés;
- le moteur utilise ces phases pour remplir `green_links` ;
- le client interagit avec les contrôleurs via l'éditeur pour modifier les durées et les compositions des phases.

Roundabouts

Fichier principal: `src/map/roundabout.rs`.

Ce chapitre couvre les helpers dédiés à la finalisation des ronds-points après leur création géométrique.

Structures

RoundaboutHandle

Handle retourné par la création d'un rond-point.

Champ	Rôle
<code>ring_node_ids</code>	Identifiants des noeuds formant l'anneau.
<code>ring_road_ids</code>	Identifiants des routes de l'anneau.

Fonction principale

`finalize_roundabout_links`

Cette fonction adapte les priorités après la création du rond-point.

Elle identifie les liens d'entrée venant de l'extérieur du cercle et les transforme en mouvements de type `Yield`. Elle parcourt ensuite les routes de l'anneau pour marquer comme `Yield` les `foe_links` en conflit avec ces entrées, ce qui permet de modéliser correctement la priorité d'insertion dans le rond-point.

Lecture fonctionnelle

- le handle conserve les identifiants créés par l'éditeur;
- `finalize_roundabout_links` applique les règles de priorité après la géométrie brute;
- le module n'ajoute pas les noeuds lui-même, il complète un rond-point déjà construit.

Link IDs & how to obtain them

Ce chapitre explique comment récupérer les identifiants de liens internes et pourquoi ils sont indispensables pour les feux et les mouvements de carrefour.

Principe

`build_intersections(&mut map)` génère les `InternalLane` et les `Link` de chaque carrefour. C'est à ce moment que les identifiants de lien sont attribués et qu'ils deviennent utilisables par le moteur et par les contrôleurs de feux.

Workflow conseillé

1. Finaliser complètement la topologie de la carte.
2. Appeler `build_intersections(&mut map)`.
3. Inspecter la structure de la carte côté serveur pour relever les identifiants de `InternalLane` et de `Link` utiles.
4. Construire les phases de feux ou les priorités de rond-point à partir de ces identifiants.

Ce qu'il faut observer

- Les identifiants de liens sont stables tant que la topologie ne change pas.
- Toute modification structurelle de la carte peut entraîner une régénération des identifiants.
- Les paquets de sérialisation grand public n'exposent pas directement ces identifiants, donc l'inspection doit se faire côté serveur.

Usage pratique

Les identifiants de liens servent principalement à:

- configurer des phases de feux tricolores;
- vérifier les conflits dans les intersections;
- comprendre quel mouvement le moteur a prévu dans le `DrivePlan` d'un véhicule.

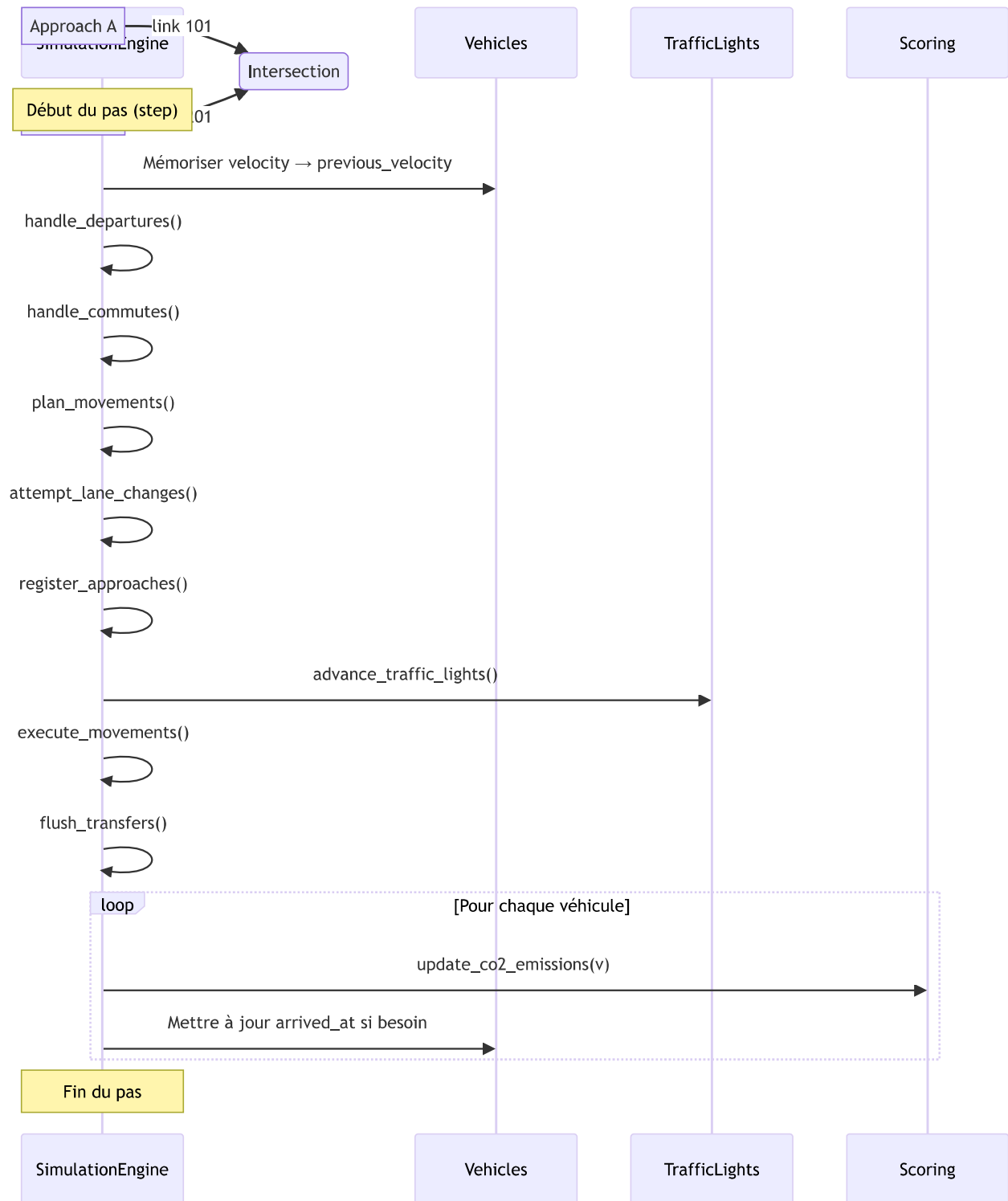
Intersection scenarios (deadlock, priority-right, traffic-light)

This page describes small scenarios you can construct to validate engine behavior.

Scenario 1 — Priority-right deadlock (2-vehicle deadlock)

- Setup: two approaches A and B arriving simultaneously, both are `yield` to each other (e.g. misconfigured foe links), both wait for the other → deadlock.
- Expected behavior: engine should detect no progress; remedies include adding small jitter to arrival times or an impatience escalation that forces one vehicle to proceed after timeout.

Mermaid diagram (schematic):



Scenario 2 — Priority-right example

- Setup: Ego on approach A, Foe on approach B. Foe arrives earlier or is to the right (depending on geometry). If `foe_is_to_the_right` returns true, Ego should yield.
- Test: tune arrival windows to make tie-breaker rely on `foe_is_to_the_right`.

Scenario 3 — Traffic-light timing

- Setup: traffic light with two phases: phase 1 opens links [101,102] for 10s, phase 2 opens [201] for 8s.
- Observations: `advance_traffic_lights` should populate `green_links` with ids of currently green links; `is_link_open` should allow `TrafficLight` type links only when

their id is in `green_links`.

How to run scenarios (suggestion)

1. Construct a small `Map` programmatically via `editor::add_node / add_road` or by starting from an example map.
2. Call `build_intersections(&mut map)`.
3. Create `Vehicle` instances with `trip` origins/destinations designed to traverse the target links and set `departure_time` to control arrival ordering.
4. Run `SimulationEngine::new(config, vehicles)` and step through `engine.step()` while logging `DrivePlanEntry` and `link_states`.

Map editor examples

Cette page donne des repères d'utilisation pour le module d'édition sans recopier le code source.

Ajouter un rond-point

Une création de rond-point typique repose sur un centre, un rayon et un nombre de bras suffisant pour laisser de la place aux branches. Le handle retourné permet ensuite de connecter les routes d'accès extérieures à chaque noeud de l'anneau.

Ajouter un contrôleur de feux

Un contrôleur est décrit par une liste de phases. Chaque phase liste les liens autorisés et les durées de vert et de jaune. Les liens utilisés doivent être valides et être récupérés après reconstruction des intersections.

Édition via le WebSocket

Les opérations de base disponibles côté client sont l'ajout, la suppression et la mise à jour de noeuds et de routes. Les helpers de plus haut niveau, comme la création de ronds-points ou de feux, restent des opérations serveur ou de démarrage de carte.

Bon réflexe

- construire la topologie;
- reconstruire les intersections;
- récupérer les identifiants de liens;
- seulement ensuite configurer les feux ou les ronds-points avancés.

Simulation overview

Le sous-système `simulation` rassemble tout ce qui transforme une carte routière en exécution dynamique.

Il se décompose en quatre chapitres:

- [Configuration](#) : paramètres globaux d'une simulation et constantes de réglage.
- [Engine details](#) : ordonnancement des véhicules, gestion des feux et progression temporelle.
- [Kinematics helpers](#) : formules de vitesse, d'arrivée et de sortie des voies.
- [Vehicle model](#) : structure d'un véhicule, trajet, état courant et profil physique.

En pratique, la carte fournit la topologie, le véhicule fournit l'état individuel, la cinématique fournit les bornes physiques, et le moteur orchestre l'ensemble à chaque pas de temps.

Simulation configuration

Fichier: `src/simulation/config.rs`.

Ce chapitre décrit les paramètres qui contrôlent le comportement global du moteur de simulation, les poids du score et les constantes partagées entre les algorithmes de déplacement.

Structures

ScoreWeights

Pondérations appliquées au score final. Elles sont copiées depuis les réglages de la carte afin que le score reste cohérent avec la configuration de projet.

Champ	Rôle
<code>time</code>	Favorise les trajets rapides.
<code>success</code>	Favorise les véhicules arrivés à destination.
<code>pollution</code>	Favorise une faible émission de CO2.
<code>infrastructure</code>	Favorise un réseau routier compact.

La méthode `ScoreWeights::from_settings` lit ces coefficients directement dans les réglages de la carte (`MapSettings`).

SimulationConfig

Configuration concrète passée au moteur.

Champ	Rôle
<code>start_time</code>	Temps simulé de départ.
<code>end_time</code>	Temps simulé de fin.
<code>time_step</code>	Pas de temps de la boucle de simulation.
<code>minimum_gap</code>	Distance minimale entre deux véhicules consécutifs sur une voie.
<code>map</code>	Carte active utilisée par le moteur.
<code>score_weights</code>	Pondérations utilisées lors du calcul final du score.

La méthode `SimulationConfig::new` construit une configuration prête à l'emploi en partant d'une carte et en appliquant les poids stockés dans `MapSettings`.

Constantes de réglage

Constante	Rôle
MAX_SPEED	Borne supérieure générale utilisée par les accélérations et la sélection de vitesse.
ACCELERATION_EXPONENT	Exposant utilisé dans la loi de type IDM pour lisser la montée en vitesse.
LOOK_AHEAD	Fenêtre de prudence temporelle utilisée pour anticiper les conflits d'intersection.
STOP_DWELL_TIME	Durée minimale d'attente avant de repartir après un arrêt complet.
IMPATIENCE_RATE	Vitesse de croissance de l'impatience d'un véhicule bloqué.
MIN_CREEP_SPEED	Vitesse de reptation appliquée dans les situations très lentes.
LANE_WIDTH	Largeur nominale d'une voie pour le rendu et l'offset latéral.

Lecture rapide

- `ScoreWeights` dit comment interpréter les critères du score.
- `SimulationConfig` dit quand la simulation commence et s'arrête, à quel rythme elle avance, et à quelle distance les véhicules doivent se tenir.
- Les constantes définissent des ordres de grandeur communs entre le moteur, le calcul d'accélération et la logique d'intersection.

Engine details

Fichier: `src/simulation/engine.rs`.

Le moteur est le cœur de la simulation. Il maintient les véhicules, fait avancer le temps, arbitre les conflits d'intersection, gère les feux tricolores et agrège les données nécessaires au score final.

Le trait `Simulation`

Le trait formalise l'interface minimale d'un moteur de simulation:

- `new(config, vehicles)` construit une instance prête à tourner.
- `run()` exécute la simulation complète jusqu'à la fin.
- `step()` avance d'un pas de temps.
- `get_score()` calcule le score final.

Structures internes

`PendingTransfer`

Représente un déplacement temporaire d'un véhicule entre deux voies. Cette structure sert à différer certaines mutations du graphe de véhicules jusqu'à la fin du pas de temps, afin d'éviter les conflits d'emprunts et de conserver une exécution déterministe.

`TrafficLightRuntimeState`

Stocke l'état courant d'un feu tricolore pendant l'exécution:

- `phase_index` : phase active actuelle.
- `time_in_phase` : temps déjà passé dans cette phase.

`BusLine`

Représente une ligne de bus active dans la simulation.

- `id` : identifiant de la ligne.

- `name` : nom lisible de la ligne.
- `stop_node_ids` : liste des arrêts.
- `vehicle_id` : identifiant du bus associé à la ligne.

SimulationEngine

Structure principale du moteur.

Champ	Rôle
<code>config</code>	Configuration globale de la simulation.
<code>vehicles</code>	Liste complète des véhicules.
<code>current_time</code>	Temps simulé courant.
<code>vehicles_by_lane</code>	Index des véhicules par voie, triés du plus proche au plus loin.
<code>link_states</code>	Réservations temporelles par lien d'intersection.
<code>all_vehicles_arrived</code>	Indique si tous les véhicules ont terminé.
<code>green_links</code>	Identifiants des liens actuellement ouverts par les feux.
<code>bus_lines</code>	Lignes de bus actives.
<code>next_bus_line_id</code>	Compteur de génération des lignes de bus.
<code>next_vehicle_id</code>	Compteur de génération des véhicules supplémentaires.
<code>pending_transfers</code>	Transferts différés à appliquer en fin de pas.
<code>traffic_light_states</code>	État runtime de chaque contrôleur de feux.
<code>link_directory</code>	Index global des <code>Link</code> du graphe pour accès rapide.

Cycle de simulation

SimulationEngine::new

Crée un moteur à partir d'une configuration et d'une liste de véhicules. L'instance initialise les états de feux à partir de la carte, construit un index de tous les liens du réseau, et démarre le temps simulé à `start_time`.

SimulationEngine::run

Prépare d'abord tous les chemins des véhicules, puis appelle `step()` jusqu'à atteindre `end_time`.

SimulationEngine::step

Un pas de simulation suit toujours le même ordre:

1. mémoriser la vitesse précédente de chaque véhicule,
2. gérer les départs,
3. préparer les mouvements,
4. enregistrer les approches,
5. faire avancer les feux,
6. exécuter les déplacements,
7. appliquer les transferts différés,
8. mettre à jour les émissions et les dates d'arrivée.

Changements de voie (Modèle LC2013)

Le moteur implémente une variante du modèle LC2013 pour gérer les changements de voie sur les routes multi-voies. Un changement de voie est tenté à chaque pas via `attempt_lane_changes`.

Stratégies

- **Keep Right** : Les véhicules tendent à se rabattre sur la voie la plus à droite si elle est libre et sécurisée.
- **Overtaking** : Un véhicule tente de déboîter vers la gauche s'il est bloqué par un véhicule plus lent et que la voie de gauche offre un avantage de vitesse.

Conditions de sécurité

Pour qu'un changement de voie soit validé, le moteur vérifie :

- **Gap Front/Rear** : Un espace minimal (`LC2013_SAFE_GAP_FRONT` et `LC2013_SAFE_GAP_REAR`) doit être disponible sur la voie cible.
- **Sécurité du suiveur** : Le nouveau suiveur sur la voie cible ne doit pas avoir à freiner trop brutalement (`LC2013_MAX_DECEL_FOLLOWER`).

- **Cooldown** : Un délai (`LC2013_COOLDOWN`) est imposé entre deux changements de voie successifs pour éviter les oscillations.

Méthodes de cycle détaillées

handle_departures

Gère le passage des véhicules de l'état `WaitingToDepart` à `OnRoad`.

- **Entrée** : `&mut self`.
- **Action** : Vérifie pour chaque véhicule en attente si son heure de départ est arrivée et si la première voie de son trajet est libre. Si oui, insère le véhicule sur la voie.

handle_commutes

Gère les étapes des plans de trajets pendulaires (`CommutePlan`).

- **Action** :
 1. Injecte les véhicules pour le trajet aller.
 2. Surveille l'arrivée des véhicules aller pour programmer le trajet retour (heure actuelle + temps d'attente).
 3. Injecte les véhicules retour à l'heure prévue.

plan_movements

Phase de décision pour tous les véhicules actifs.

- **Action** : Parcourt les véhicules sur les routes normales et recalcule leur `drive_plan` s'ils s'approchent d'une intersection.

rebuild_drive_plan

Calcule les réservations pour les prochaines intersections du trajet d'un véhicule.

- **Entrées** : `vidx: usize`.
- **Action** : Simule le mouvement futur du véhicule sur plusieurs segments pour estimer les heures d'arrivée aux carrefours et choisir les voies internes optimales.

register_approaches

Enregistre officiellement les intentions de passage dans les carrefours.

- **Action** : Pour chaque véhicule ayant un plan de conduite, ajoute son `ApproachData` à l'état du lien correspondant (`link_states`) si le passage est autorisé.

advance_traffic_lights

Fait évoluer le temps interne des contrôleurs de feux.

- **Entrée** : `&mut self`.
- **Action** : Incrémente le compteur de chaque feu. Si la durée d'une phase est dépassée, passe à la suivante et met à jour les liens ouverts (`green_links`).

execute_vehicle

Applique la physique du mouvement à un véhicule.

- **Entrées** : `vidx: usize, lane_id: LaneId`.
- **Action** :
 1. Détermine la vitesse de sécurité selon le véhicule de devant et l'état des liens.
 2. Calcule l'accélération (IDM).
 3. Met à jour la position et la vitesse.
 4. Gère les transitions entre les routes et les intersections (via `process_lane_advances`).

determine_safe_speed

Calcule la vitesse maximale autorisée pour un véhicule en tenant compte des obstacles.

- **Entrées** : `vidx: usize`.
- **Action** : Analyse la distance jusqu'au prochain véhicule sur la même voie et l'état de l'intersection à venir (Stop, feu rouge, ou conflit).
- **Retour** : `(v_cible, distance_obstacle)`.

find_link

Cherche un lien d'intersection par son ID.

- **Retour** : `Option<Link>`.

vehicle_ahead_info

Identifie le véhicule précédant immédiatement un autre sur une voie donnée.

- **Retour** : (distance, vitesse) .

process_lane_advances

Gère le passage d'un véhicule d'une section à la suivante (ex: sortie de route vers intersection).

- **Action** : Détecte si le véhicule a dépassé la longueur de sa voie actuelle et déclenche les fonctions de transition appropriées.

exit_internal_lane

Gère la sortie d'un carrefour pour entrer sur une nouvelle route.

- **Action** : Met à jour la position du véhicule et prépare un transfert vers la voie de destination.

enter_junction_or_arrive

Gère l'entrée dans un carrefour ou l'arrivée à destination.

- **Action** :
 1. Si le véhicule a atteint son dernier nœud, il passe à l'état `Arrived` .
 2. Sinon, le moteur vérifie si la voie de destination après la jonction n'est pas saturée (prévention de l'empilement).
 3. Il existe une probabilité `direction_error_probability` que le véhicule choisisse une sortie aléatoire différente de celle prévue.
 4. En cas d'erreur, le véhicule recalcule son trajet (`reroute_from_junction`) et son plan de conduite.
 5. Le véhicule s'engage enfin sur la voie interne (originale ou nouvelle).

flush_transfers

Applique tous les changements de voies mis en attente pendant le pas de temps.

- **Action** : Met à jour `vehicles_by_lane` et les champs `current_lane` des véhicules concernés.

Simulation Longue Durée (24h)

Le moteur supporte des simulations s'étendant sur plusieurs heures réelles. Pour maintenir la stabilité :

- **Distribution réaliste** : Les flux de véhicules ne sont pas constants mais suivent des cycles jour/nuit et des pics pendulaires (voir [Commute Plans](#)).
- **Gestion de la mémoire** : Les véhicules arrivés sont archivés pour le scoring mais ne consomment plus de ressources physiques dans le moteur.
- **Contrôles de vitesse** : Le moteur permet d'ajuster le multiplicateur de vitesse pour accélérer l'écoulement du temps simulé.

lane_insert_sorted

Helper pour insérer un véhicule dans l'index d'une voie tout en maintenant le tri par position.

add_bus_from_saved

Ajoute un bus à la simulation à partir d'une ligne sauvegardée.

- **Entrée** : `bl: &SavedBusLine`.
- **Action** : Crée un véhicule de type `Bus`, initialise son trajet passant par tous les arrêts de la ligne, et l'ajoute à la liste des véhicules actifs.
- **Retour** : `bool` (succès).

Ce qui vaut d'être retenu

- Le moteur ne se contente pas de déplacer les véhicules: il réserve aussi les intersections à l'avance.
- Les feux, la priorité et les conflits temporels sont évalués conjointement.
- La séparation entre planification et exécution évite de mélanger les décisions de sécurité avec le déplacement physique.

Modèle de Véhicule

Le module `vehicle` définit les structures et les comportements des véhicules au sein de la simulation, incluant leur dynamique, leur planification de trajet et leur état.

Énumérations

VehicleType

Définit le type de motorisation du véhicule. Utilisé principalement pour le calcul des émissions de CO2 et du score environnemental.

- **Variantes** : `Hybride`, `Electrique`, `Essence`, `Diesel`.

VehicleKind

Définit la catégorie du véhicule.

- **Variantes** :
 - `Car` : Voiture particulière standard.
 - `Bus` : Transport en commun (plus long, vitesse et accélération différentes).

LaneId

Identifiant unique d'une voie sur laquelle se trouve un véhicule.

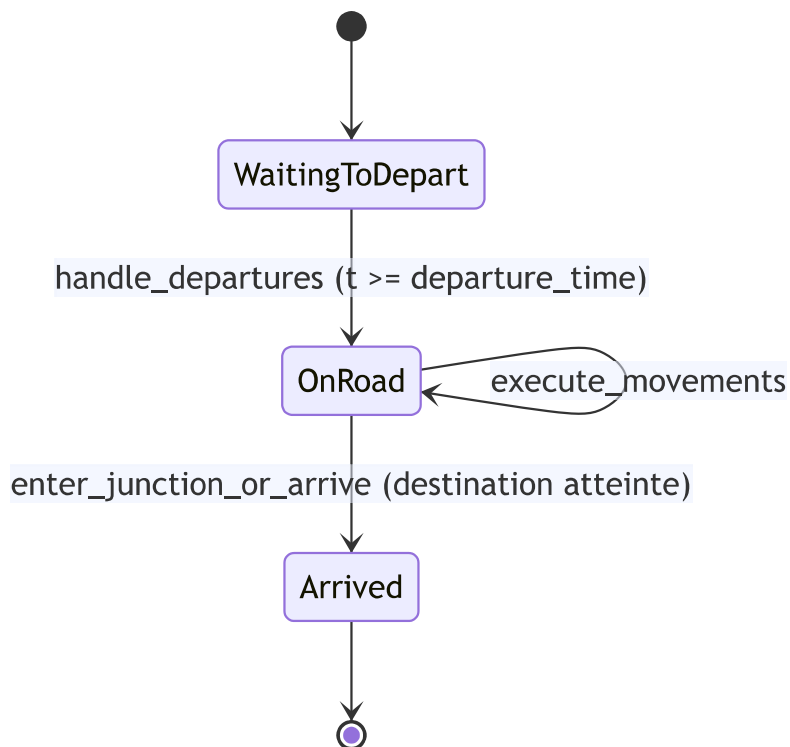
- **Variantes** :
 - `Normal(EdgeIndex, u32)` : Voie standard sur une route (Index de l'arête, Index de la voie).
 - `Internal(u32, u32)` : Voie interne à une intersection (ID de l'intersection, ID de la voie interne).

VehicleState

Représente l'état actuel du véhicule dans le cycle de simulation.

- **Variantes** :
 - `WaitingToDepart` : Le véhicule attend son heure de départ prévue.

- `OnRoad` : Le véhicule est activement en train de circuler sur la carte.
- `Arrived` : Le véhicule a atteint sa destination finale.



Structures

VehicleSpec

Spécifications physiques et performances d'un type de véhicule.

- **Champs :**

- `kind` : `VehicleKind`.
- `max_speed` : Vitesse maximale autorisée (m/s).
- `max_acceleration` : Accélération maximale (m/s²).
- `comfortable_deceleration` : Décélération confortable pour le conducteur (m/s²).
- `reaction_time` : Temps de réaction du conducteur (s).
- `length` : Longueur du véhicule (m).

TripRequest

Requête de trajet définissant le point de départ et d'arrivée.

- **Champs :**

- `origin` : Nœud de départ (`NodeIndex`).
- `destination` : Nœud d'arrivée (`NodeIndex`).

- `departure_time` : Heure de départ prévue (s).

DrivePlanEntry

Entrée dans le plan de conduite calculé par le moteur de simulation pour gérer les réservations de liens.

- **Champs** : Contient les informations de réservation de lien (`link_id`), l'heure d'arrivée prévue (`arrival_time`), la vitesse de passage (`v_pass`), etc.

Vehicle

Structure principale représentant un véhicule actif.

- **Champs principaux** :
 - `id` : Identifiant unique.
 - `spec` : Spécifications physiques.
 - `state` : État actuel.
 - `motorization` : Type de moteur (`VehicleType`).
 - `waypoints` : Liste de points de passage intermédiaires.
 - `path` : Liste complète des nœuds du trajet calculé.
 - `position_on_lane` : Distance parcourue sur la voie actuelle (m).
 - `velocity` : Vitesse actuelle (m/s).
 - `previous_velocity` : Vitesse au pas précédent (pour le calcul de l'accélération).
 - `current_lane` : Voie occupée (`Option<LaneId>`).
 - `waiting_time` : Temps passé à l'arrêt devant une intersection.
 - `impatience` : Score d'impatience croissant quand le véhicule est bloqué.
 - `emitted_co2` : Cumul du CO2 émis.
 - `distance_traveled` : Distance totale parcourue.
 - `arrived_at` : Instant précis de l'arrivée (`Option<f32>`).
 - `lane_change_cooldown` : Temps restant avant de pouvoir changer de voie à nouveau (s).
 - `desired_velocity` : Vitesse que le véhicule tente d'atteindre, influencée par la limite de vitesse et le comportement du conducteur (m/s).

Fonctions globales

fastest_path

Calcule le chemin le plus rapide entre deux points en utilisant l'algorithme A*.

- **Entrées :**
 - `map`: `&Map` : Référence à la carte.
 - `source`: `NodeIndex` : Point de départ.
 - `destination`: `NodeIndex` : Point d'arrivée.
- **Action :** Utilise la longueur des routes divisée par la vitesse limite comme poids pour les arêtes.
- **Retour :** `Option<Vec<NodeIndex>>` (liste ordonnée des nœuds du chemin).

Méthodes de Vehicle

new

Crée une nouvelle instance de véhicule.

- **Entrées :** `id`, `spec`, `trip`, `motorization`.
- **Retour :** Une instance de `Vehicle` initialisée à l'état `WaitingToDepart`.

update_path

Calcule ou met à jour le chemin complet du véhicule (incluant les points de passage forcés).

- **Entrées :** `map`: `&Map`.
- **Action :** Remplit le champ `self.path` en concaténant les segments de chemin les plus rapides. Gère les **points de passage (Waypoints)** obligatoires pour les bus afin de respecter les arrêts prévus.

compute_acceleration

Calcule l'accélération à appliquer selon le modèle IDM (Intelligent Driver Model).

- **Entrées :**
 - `desired_velocity` : Vitesse souhaitée.
 - `minimum_gap` : Distance de sécurité minimale.
 - `vehicle_ahead_distance` : Distance avec le véhicule de devant.
 - `vehicle_ahead_velocity` : Vitesse du véhicule de devant.
- **Action :**
 - Le véhicule calcule sa vitesse désirée en fonction de `spec.max_speed` et de la limite de vitesse de la route.
 - L'accélération est calculée pour équilibrer l'envie d'atteindre `desired_velocity` et le besoin de maintenir une distance de sécurité par rapport au leader.

- **Retour** : `f32` (accélération en m/s^2).

get_coordinates

Calcule les coordonnées (x, y) réelles du véhicule sur la carte.

- **Entrées** : `map`: `&Map` .
- **Action** : Interpole la position entre les nœuds de la voie actuelle, en tenant compte des décalages latéraux (lanes) et des trajectoires dans les intersections.
- **Retour** : `Coordinates` .

get_heading

Calcule l'orientation (angle) du véhicule en radians.

- **Entrées** : `map`: `&Map` .
- **Retour** : `f32` (angle entre $-\pi$ et π).

get_current_node / get_next_node

Accesseurs pour les nœuds actuels et suivants du trajet.

get_current_road

Récupère l'arête du graphe correspondant à la route actuelle.

- **Retour** : `Option<EdgeIndex>` (None si le véhicule est dans une intersection).

Plans de trajet pendulaire (Commute)

Le système de trajet pendulaire (`CommutePlan`) gère les cycles de déplacement aller-retour des travailleurs et des résidents, permettant de simuler des journées complètes de trafic réaliste.

Structure du plan

Un `CommutePlan` lie deux véhicules (ou un véhicule effectuant deux trajets) pour représenter un cycle complet :

Champ	Description
<code>id</code>	Identifiant unique du plan.
<code>outbound_vehicle_id</code>	ID du véhicule pour le trajet aller (ex: Domicile -> Travail).
<code>return_vehicle_id</code>	ID du véhicule pour le trajet retour (ex: Travail -> Domicile).
<code>outbound_departure_time_s</code>	Heure de départ prévue pour l'aller.
<code>return_waiting_time_s</code>	Temps d'attente à destination avant le retour.
<code>state</code>	État actuel du cycle (<code>OutboundPending</code> , <code>Running</code> , <code>Completed</code> , etc.).

Distribution temporelle

Pour générer un trafic réaliste, les heures de départ et les temps d'attente ne sont pas uniformes mais suivent des **lois Beta**, simulant les pics de trafic du matin et du soir.

Paramètres de distribution

- **Fenêtre de temps** : 12 heures (43 200 s) pour chaque phase.
- **Départ (Aller)** : Paramètres $\alpha=6.33$, $\beta=3.67$. Cette distribution place le pic de départ vers le milieu/fin de la matinée.
- **Attente (Travail)** : Paramètres $\alpha=7.25$, $\beta=2.75$. Simule une journée de travail standard de plusieurs heures.

Cycle de vie

1. **OutboundPending** : Le plan est créé et attend l'heure `outbound_departure_time_s`.
2. **OutboundRunning** : Le véhicule aller est injecté dans la simulation.
3. **WaitingForReturnDeparture** : Une fois l'aller arrivé, le moteur calcule l'heure de retour : `heure_arrivée + return_waiting_time_s`.
4. **ReturnRunning** : Le véhicule de retour est injecté.
5. **Completed** : Le cycle est terminé après l'arrivée du retour.

Intégration dans le moteur

Le `SimulationEngine` gère une liste de `commute_plans`. À chaque pas, il vérifie quels plans doivent passer à l'étape suivante via la méthode `handle_commutes()`.

Kinematics helpers

Fichier: `src/simulation/kinematics.rs`.

Les fonctions de ce module servent à convertir une distance restante, une vitesse courante et des bornes d'accélération en temps estimé ou en vitesse d'arrêt possible. Elles sont utilisées par le moteur pour réserver les intersections et construire les `DrivePlanEntry`.

Unités: distances en mètres, vitesses en mètres par seconde, durées en secondes.

`v_stop_at(dist, d_max)`

Cette fonction calcule la vitesse maximale qui permettrait de s'arrêter exactement sur une distance `dist` avec une décélération confortable `d_max`. La relation utilisée est celle du mouvement uniformément décéléré: $v = \sqrt{2 d_{\text{max}} \cdot \text{dist}}$.

Exemple d'ordre de grandeur: sur 25 m avec une décélération de 3 m/s², la vitesse d'arrêt théorique est d'environ 12,25 m/s.

`arrival_time(dist, v0, v1, a_max, d_max)`

Cette fonction estime le temps minimal nécessaire pour parcourir `dist` en partant de `v0` et en rejoignant `v1`, tout en respectant une accélération maximale `a_max` et une décélération maximale `d_max`.

Le moteur s'en sert pour savoir quand un véhicule atteindra une jonction et quand il la quittera. Lorsque la distance est courte, la fonction tient compte du fait qu'il n'y a pas toujours une phase de croisière: tout peut être consacré à accélérer ou à freiner.

`leave_time(t_arrive, lane_len, veh_len, v_arrive, v_leave)`

Cette fonction transforme une heure d'arrivée en heure de sortie estimée, en considérant la longueur utile de la voie, la longueur du véhicule et une vitesse moyenne bornée. Elle produit une fenêtre temporelle plus réaliste qu'un simple instant ponctuel, ce qui évite de faire se chevaucher artificiellement plusieurs véhicules dans une même intersection.

approach_speed(link_type, road_speed_limit)

Cette fonction choisit la vitesse d'approche cible d'un lien selon sa priorité:

- un lien prioritaire conserve la vitesse de la route,
- un lien en cédez-le-passage est volontairement plus lent,
- un arrêt impose une vitesse nulle,
- un feu tricolore se comporte comme une route normale tant qu'il est vert.

Ce qu'il faut retenir

- `v_stop_at` répond à la question: "à quelle vitesse dois-je être pour pouvoir m'arrêter à temps?"
- `arrival_time` répond à la question: "quand vais-je arriver au carrefour?"
- `leave_time` répond à la question: "quand aurai-je complètement dégagé le carrefour?"
- `approach_speed` répond à la question: "à quelle vitesse dois-je aborder ce lien?"

DrivePlanEntry & LinkState (JSON examples)

This page shows the runtime structures produced by the engine and how they look conceptually as JSON. These are not directly serialized by the server as a public API, but they describe the internal data the engine uses.

DrivePlanEntry (fields)

- `link_id` (u32): id of the internal movement (see `map/link_ids.md`).
- `lane_id` (LaneId): origin lane (Normal(edge, lane_idx) or Internal(junction, il_id)).
- `via_internal_lane_id` (u32): internal lane id traversed at the junction.
- `junction_id` (u32): intersection id.
- `v_pass` (f32): pass velocity (m/s).
- `v_wait` (f32): wait velocity if blocked (m/s).
- `arrival_time`, `leave_time` (f32): estimated times in simulation seconds.
- `arrival_speed`, `leave_speed` (f32): speeds used for timing.
- `distance` (f32): cumulative distance from current vehicle position to link.
- `set_request` (bool): whether the vehicle registers an approach request for this link.

Example DrivePlanEntry JSON (single entry):

```
{
  "link_id": 101,
  "lane_id": { "Normal": [ 42, 0 ] },
  "via_internal_lane_id": 55,
  "junction_id": 10,
  "v_pass": 9.5,
  "v_wait": 3.0,
  "arrival_time": 12.34,
  "leave_time": 12.90,
  "arrival_speed": 8.5,
  "leave_speed": 7.0,
  "distance": 45.2,
  "set_request": true
}
```

LinkState and ApproachData

`LinkState` holds approaching vehicles keyed by vehicle id. `ApproachData` contains estimated arrival/leave times and speeds.

Example LinkState JSON:

```
{
  "link_id": 101,
  "approaching": {
    "42": { "arrival_time": 12.34, "leave_time": 12.9, "arrival_speed": 8.5,
"leave_speed": 7.0, "will_pass": true },
    "1001": { "arrival_time": 13.5, "leave_time": 14.1, "arrival_speed": 9.0,
"leave_speed": 8.0, "will_pass": false }
  }
}
```

How the engine uses these structures

- After `rebuild_drive_plan`, vehicles with `set_request=true` insert `ApproachData` into `link_states[link_id].approaching`.
- `is_link_open` consults `link_states` and `foe_links` to decide whether the ego vehicle may proceed.

Debug tip: to reproduce engine decisions, record the `DrivePlanEntry` list for the ego vehicle and the `LinkState.approaching` map for all `foe_links`, then evaluate `time_window_conflict` on arrival/leave intervals.

Scoring system

Fichier: `src/scoring/mod.rs`.

Le module de scoring transforme l'état final d'une simulation en un nombre unique et lisible. Il compare les véhicules arrivés à des références théoriques pour le temps, le CO2 et la taille du réseau.

Constantes physiques et de calibration

Constante	Rôle
AIR_DENSITY	Densité de l'air utilisée pour l'effort aérodynamique.
GRAVITY	Accélération de la pesanteur.
DRIVE_TRAIN_EFFICIENCY	Rendement global de la transmission mécanique.
FRANCE_GRID_CO2_PER_J	Équivalent CO2 par joule électrique pour le mix français.

Structures internes et publiques

KindParams

Paramètres physiques dépendant du type de véhicule.

- masse
- coefficient de traînée
- surface frontale
- résistance au roulement
- puissance consommée à l'arrêt

Ces paramètres permettent d'estimer de façon cohérente la consommation et les émissions.

MinHeap

Petit wrapper de file de priorité utilisé lors du calcul de l'arbre couvrant minimal approché. Il sert uniquement à faire avancer l'algorithme de coût minimal sur les points de la carte.

Score

Résultat complet envoyé au client à la fin de la simulation.

Champ	Rôle
score	Valeur finale synthétique.
total_trip_time	Temps réellement consommé par les véhicules arrivés.
ref_total_trip_time	Temps de référence théorique minimal.
total_emitted_co2	CO2 réellement produit.
ref_total_emitted_co2	CO2 de référence minimal.
network_length	Longueur totale des routes distinctes.
ref_network_length	Borne inférieure du meilleur réseau possible.
success_rate	Part des véhicules arrivés.

Fonctions de calcul

kind_params

Fournit les paramètres physiques pour une catégorie de véhicule donnée.

- **Entrées** : `kind: VehicleKind`.
- **Retour** : Structure `KindParams`.

emission_params

Retourne les paramètres d'émission pour un couple (motorisation, catégorie).

- **Entrées** : `motorization: VehicleType`, `kind: VehicleKind`.
- **Retour** : (`f32`, `f32`) représentant (CO2 par Joule, CO2 par seconde au ralenti).

update_co2_emissions

Calcule et ajoute le CO2 produit par un véhicule durant un pas de temps.

- **Entrées** :
 - `vehicle: &mut Vehicle` : Le véhicule à mettre à jour.
 - `time_step: f32` : Durée du pas de temps.
- **Action** : Calcule l'effort de traction (résistance air, roulement, accélération) puis la consommation énergétique et les émissions correspondantes.

- **Post-conditions** : `vehicle.emitted_co2` est incrémenté.

get_minimal_time_travel_by_road / get_minimal_co2_by_road

Calculent les minima théoriques pour un véhicule sur une route donnée sans trafic ni contrainte.

- **Entrées** : `map`, `road_index`, `motorization`, `kind`.
- **Retour** : `f32`.

get_vehicle_min_time / get_vehicle_min_co2

Estiment le temps et les émissions minimums pour l'intégralité du trajet d'un véhicule.

- **Action** : Somment les minima de chaque route composant le chemin du véhicule.
- **Retour** : `f32`.

euclidean

Calcul de la distance Euclidienne en 64 bits.

- **Retour** : `f64`.

mst_length

Calcule la longueur totale de l'arbre couvrant minimal pour un nuage de points (algorithme de Prim).

- **Retour** : `f64`.

steiner_lower_bound

Estime la longueur minimale théorique nécessaire pour connecter tous les points d'intérêt (habitations et lieux de travail).

- **Entrées** : `map`: `&Map`.
- **Action** : Calcule l'Arbre Couvrant Minimal (MST) sur l'ensemble des points d'intérêt, puis applique un coefficient correctif de $\frac{\sqrt{3}}{2} \approx 0.866$ pour approcher la borne inférieure de Steiner.
- **Retour** : `f64` (longueur en mètres).

compute_score

Calcule le score final synthétique de la simulation.

- **Entrées :**
 - `vehicles: &[Vehicle]` : Liste de tous les véhicules.
 - `config: &SimulationConfig` : Configuration incluant les poids du score.
- **Action :**
 1. Agrège les temps de trajet et émissions réelles de tous les véhicules.
 2. Calcule les références théoriques (minima).
 3. Calcule le taux de réussite (véhicules arrivés / total).
 4. Combine les ratios (Ref/Réel) pondérés par les poids de la configuration.
- **Retour :** Structure `Score` .

Lecture du score

Le score final compare toujours l'exécution réelle à une exécution de référence:

- si le trajet est rapide, le terme temps augmente;
- si beaucoup de véhicules arrivent, le terme succès augmente;
- si le CO2 réel est proche du CO2 minimal, le terme pollution augmente;
- si le réseau est compact, le terme infrastructure augmente.

Le résultat final est ensuite envoyé au client sous forme de `ServerPacket::Score` .

Build & Run

Prerequis:

- Rust toolchain (cargo)
- `mdbook` pour construire la documentation (optionnel localement)

Construire la documentation mdBook (depuis le dossier `server`):

```
cd server/doc
mdbook build
```

Pour lancer le serveur localement (port 8080):

```
# depuis la racine du repo
cd server
cargo run --release
```

Créer une instance de simulation via HTTP POST `/api/simulations` puis se connecter en WebSocket sur `/ws?uuid=<uuid>&token=<token>` .

Commit & push de la documentation:

```
git add server/doc
git commit -m "Add generated mdBook documentation for backend"
git push origin feature/documentation
```